



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

**Dipartimento di Ingegneria e Scienze dell'Informazione
e Matematica**

Tesi di Laurea in Informatica

ROOC: linguaggio ed ambiente integrato per la modellazione e soluzione di problemi di ottimizzazione lineare

Relatore

Prof. Matteo Spezialetti

Correlatore

Prof. Stefano Smriglio

Laureando

Enrico Menichelli

279061

Anno Accademico 2025/2026

*A tutti gli studenti, con l'augurio che
possano costruire strumenti migliori di quelli
che hanno trovato.*



Abstract

L'ottimizzazione matematica permette di affrontare numerosi problemi decisionali in ambito industriale, logistico ed economico, ma tra la formulazione di un problema e la sua risoluzione si colloca la *modellazione*: la traduzione del problema in un modello matematico trattabile da un risolutore. Gli strumenti esistenti per questa attività sono spesso commerciali, legati a un linguaggio di programmazione ospite o poco orientati all'apprendimento. Nell'ecosistema del linguaggio Rust, inoltre, nessuno di essi consente di esprimere vincoli logici in modelli di programmazione lineare intera mista, linearizzandoli automaticamente.

Questa tesi presenta ROOC, un linguaggio dichiarativo per la modellazione di problemi di ottimizzazione lineare e l'ambiente che lo accompagna. ROOC adotta una notazione vicina a quella matematica e separa la struttura del modello dai dati; a partire da tale descrizione si comporta come un *compilatore*, traducendo il modello di alto livello in una forma lineare standard attraverso una sequenza di trasformazioni ispezionabili.

Il lavoro si concentra sul motore di ROOC. I contributi principali sono la progettazione del linguaggio, dotato di un sistema di tipi e di costrutti di alto livello quali iteratori, funzioni e tipi di dato strutturati; la realizzazione di un compilatore multi-fase che riduce il modello alla forma standard; l'estensione del solver `microlp`, ottenuto come *fork* del crate archiviato `minilp` e ampliato dal solo semplice al supporto della programmazione lineare intera mista; e la distribuzione del motore come libreria riutilizzabile e come piattaforma web a fini didattici, eseguita interamente nel browser tramite la compilazione in WebAssembly.

Parole chiave: ottimizzazione, linguaggio di modellazione, compilatori, programmazione lineare, MILP, semplice, WebAssembly, Rust, fluent API.

Indice

Elenco delle figure	v
Elenco delle tabelle	vi
1 Introduzione	1
1.1 Contesto e motivazione	1
1.2 Il problema e l'idea di ROOC	1
1.3 Obiettivi e contributi	2
2 Background e stato dell'arte	3
2.1 Programmazione lineare e MILP	3
2.2 I modelli di ottimizzazione	4
2.3 Linguaggi e strumenti di modellazione esistenti	4
2.4 Cenni di teoria dei linguaggi e dei compilatori	6
2.5 Posizionamento di ROOC	6
3 Il linguaggio ROOC	8
3.1 Principi di design	8
3.2 Struttura di un modello	8
3.3 Esempio guida	9
3.4 Primitive e tipi di dato	10
3.5 Iteratori, <i>block functions</i> e destructuring	11
3.6 Operatori e definizione delle variabili	12
3.7 Libreria standard e funzioni definite dall'utente	13
4 Architettura del compilatore	15
4.1 La pipeline multi-fase	15
4.2 L'astrazione <i>Pipe</i>	15
4.3 Estensibilità e visualizzazione passo-passo	17
5 Analisi sintattica e rappresentazione intermedia	18
5.1 Concetti teorici	18
5.2 Applicazione in ROOC: la grammatica PEST	19
5.3 Il PreModel e l'Intermediate Language	20
5.4 Gestione e tracciamento degli errori	21
6 Analisi semantica: tipi e trasformazione del modello	22
6.1 Concetti teorici	22
6.2 Applicazione in ROOC: il sistema dei tipi	23
6.3 Il type checker	23
6.4 Trasformazione PreModel $\rightarrow$ Model	24

7	Linearizzazione e forma standard	27
7.1	Concetti teorici	27
7.2	Applicazione in ROOC: da Model a LinearModel	28
7.3	Linearizzazione di minimo, massimo e valore assoluto	29
7.4	Linearizzazione degli operatori logici	29
7.5	Trasformazione in forma standard	31
8	Risoluzione dei modelli	32
8.1	Concetti teorici	32
8.2	Applicazione in ROOC: il simplesso a due fasi	32
8.3	microlp: fork ed estensione a MILP	33
8.4	Backend esterni	34
9	Dalla libreria alla piattaforma web	35
9.1	La libreria Rust e la fluent API	35
9.2	Compilazione in WebAssembly e la libreria TypeScript	37
9.3	L'IDE web e l'LSP nel browser	38
9.4	Valore didattico	39
10	Conclusioni e sviluppi futuri	42
10.1	Riepilogo dei contributi	42
10.2	Limitazioni	42
10.3	Sviluppi futuri	43
	Bibliografia	44
A	Un esempio di compilazione: il problema della dieta	46
A.1	Sorgente	46
A.2	Modello	47
A.3	Modello lineare	48
A.4	Forma standard	48

Elenco delle figure

4.1	La pipeline di compilazione completa, dalla sorgente alla soluzione ottenuta tramite il simplesso passo-passo. Ogni nodo è una rappresentazione intermedia; ogni arco è la <i>pipe</i> che effettua la trasformazione.	16
9.1	L'editor della piattaforma web: a sinistra l'area di scrittura del modello, a destra il pannello con la pipeline di risoluzione, configurabile come sequenza di stadi.	39
9.2	Le rappresentazioni intermedie esposte dopo l'esecuzione, dal modello alla forma standard.	40
9.3	La risoluzione passo-passo del simplesso e la soluzione finale del problema della dieta.	41

Elenco delle tabelle

4.1	Le pipe disponibili e i tipi di dato che trasformano.	17
5.1	Gli operatori della grammatica PEG/PEST utilizzati in ROOC.	19

Capitolo 1

Introduzione

1.1 Contesto e motivazione

L'ottimizzazione matematica è uno strumento fondamentale per affrontare problemi decisionali in ambito industriale, logistico ed economico: dalla pianificazione della produzione all'allocazione di risorse, molte situazioni reali possono essere ricondotte alla ricerca della soluzione migliore tra molte alternative, nel rispetto di un insieme di vincoli. La *Ricerca Operativa* e l'*Ottimizzazione Combinatoria*, da cui peraltro deriva il nome del progetto ROOC, studiano proprio i metodi per formulare e risolvere problemi di questo tipo.

Tra la formulazione di un problema e la sua risoluzione si colloca però un'attività non banale: la *modellazione*, cioè la traduzione del problema in un modello matematico formale che un risolutore (*solver*) sia in grado di trattare. Questa traduzione richiede competenze specialistiche e strumenti adeguati, e gli ambienti esistenti, pur potenti, sono spesso commerciali, legati a un linguaggio di programmazione ospite o poco orientati all'apprendimento.

A queste considerazioni se ne aggiunge una legata all'ecosistema di *Rust* [12], un linguaggio in crescita, da diversi anni il più apprezzato dagli sviluppatori secondo il sondaggio annuale di Stack Overflow [17]. Uno dei suoi punti di forza è la portabilità, dato che lo stesso sorgente può essere compilato per molte piattaforme diverse, compreso il browser tramite WebAssembly. Perché un'applicazione ne benefici, però, ogni sua dipendenza deve essere *pure Rust*, cioè scritta interamente in Rust e priva di dipendenze native. Gli strumenti di modellazione più diffusi non soddisfano questo requisito, perché richiedono un runtime esterno, come l'interprete Python per Pyomo o l'eseguibile di AMPL, oppure librerie native precompilate, come nel caso di Gurobi. D'altra parte, al momento della scrittura nessuno degli strumenti di modellazione nati nell'ecosistema Rust offre vincoli logici su modelli misti né la loro linearizzazione automatica (Sezione 2.3).

1.2 Il problema e l'idea di ROOC

ROOC nasce per rendere più accessibile l'attività di modellazione e colmare questa lacuna dell'ecosistema Rust.¹ È un linguaggio dichiarativo, con una notazione vicina a quella matematica, che consente di descrivere un modello di ottimizzazione separandone la struttura dai dati. A partire da tale descrizione, ROOC si comporta come un *compilatore*: attraverso una sequenza di

¹Il codice sorgente del progetto è disponibile pubblicamente all'indirizzo <https://github.com/Specy/rooc>.

trasformazioni traduce il modello di alto livello in una forma lineare standard, che viene poi affidata a un solver per ottenere la soluzione ottima.

L'idea di fondo è dunque applicare le tecniche dei compilatori al dominio dell'ottimizzazione, ottenendo uno strumento che non solo risolve i modelli, ma ne rende ispezionabile ogni fase intermedia. Questa trasparenza, unita all'esecuzione interamente nel browser, conferisce a ROOC anche una valenza didattica.

Il motore è inoltre pensato per essere incorporato in altre applicazioni. È *pure Rust*, compresi i solver predefiniti, e oltre al linguaggio testuale offre una *fluent API*, con cui un modello si costruisce direttamente nel codice Rust. Può quindi essere compilato per qualunque target supportato dal compilatore Rust, e la piattaforma web, eseguita interamente nel browser, ne è una dimostrazione.

1.3 Obiettivi e contributi

Il presente lavoro si concentra sul motore di ROOC, e in particolare sulla progettazione del linguaggio e del relativo compilatore. I contributi principali sono:

- la progettazione di un *linguaggio* di modellazione dotato di un sistema di tipi e di costrutti di alto livello, quali iteratori, *block function* e tipi di dato strutturati (Capitolo 3);
- la realizzazione di un *compilatore multi-fase*, che traduce il modello fino alla forma standard, organizzato secondo un'architettura a *pipe* che rende ogni fase componibile e ispezionabile (Capitoli 4–7);
- l'estensione del solver `microlp`, ottenuto come *fork* del crate archiviato `minilp` e ampliato dal solo semplice al supporto delle variabili intere e binarie, fino a coprire la programmazione lineare intera mista (Capitolo 8);
- la distribuzione del motore, pure Rust insieme ai suoi solver predefiniti, come libreria Rust, che accanto al linguaggio offre una *fluent API* per costruire i modelli direttamente nel codice, come pacchetto TypeScript precompilato e come piattaforma web a fini didattici (Capitolo 9).

Capitolo 2

Background e stato dell'arte

Questo capitolo fornisce le basi necessarie a comprendere il resto della tesi: i concetti fondamentali della programmazione lineare e della sua estensione intera, i principali linguaggi e strumenti di modellazione esistenti e i concetti di teoria dei compilatori applicati nei capitoli successivi. Si delinea infine il posizionamento di ROOC rispetto a tale panorama.

2.1 Programmazione lineare e MILP

La *programmazione lineare* (*Linear Programming*, LP) è il ramo dell'ottimizzazione matematica che studia la minimizzazione (o massimizzazione) di una funzione obiettivo lineare soggetta a un insieme di vincoli lineari. Un problema di programmazione lineare può essere espresso nella forma generale

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b, \\ & x \geq 0, \end{aligned} \tag{2.1}$$

dove $x \in \mathbb{R}^n$ è il vettore delle *variabili decisionali*, $c \in \mathbb{R}^n$ il vettore dei coefficienti della funzione obiettivo, $A \in \mathbb{R}^{m \times n}$ la matrice dei coefficienti dei vincoli e $b \in \mathbb{R}^m$ il vettore dei termini noti. L'insieme dei punti che soddisfano tutti i vincoli prende il nome di *regione ammissibile*; quando non è vuota e la funzione obiettivo è limitata sulla regione, esiste almeno una *soluzione ottima*, che il problema si propone di determinare [2, 3].

Un risultato classico della teoria afferma che, se esiste una soluzione ottima, allora ne esiste una in corrispondenza di un *vertice* del poliedro che descrive la regione ammissibile. Su questa proprietà si fonda il *metodo del simplesso*, che esplora i vertici della regione spostandosi di volta in volta verso quello con valore migliore della funzione obiettivo. Il simplesso opera su una rappresentazione del problema detta *forma standard*,

$$\min_x \quad c^\top x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \tag{2.2}$$

in cui tutti i vincoli sono uguaglianze e tutte le variabili sono non negative. La conversione di un problema dalla forma (2.1) alla forma (2.2) è un passaggio puramente meccanico, che verrà trattato nel Capitolo 7 come una delle fasi della pipeline di compilazione di ROOC.

In molte applicazioni reali le variabili decisionali non possono assumere valori frazionari: rappresentano scelte indivisibili (quanti veicoli acquistare) o decisioni di tipo sì/no (se attivare o meno un impianto). Si parla allora di *programmazione lineare intera mista* (*Mixed-Integer Linear Programming*, MILP), in cui si aggiunge il vincolo di integralità su un sottoinsieme $I \subseteq \{1, \dots, n\}$ delle variabili:

$$\min_x c^\top x \quad \text{s.t.} \quad Ax \leq b, \quad x \geq 0, \quad x_j \in \mathbb{Z} \quad \forall j \in I. \quad (2.3)$$

Un caso particolarmente importante è quello delle variabili *binarie*, in cui $x_j \in \{0, 1\}$, usate per modellare decisioni logiche. L'aggiunta del vincolo di integralità cambia radicalmente la natura del problema: mentre la programmazione lineare continua è risolvibile in tempo polinomiale, la programmazione lineare intera è in generale *NP-hard* [14, 19]. La tecnica più diffusa per risolverla è il *branch-and-bound*, ripreso nel Capitolo 8, dove se ne descrive l'impiego nell'estensione del solver *microlp* al caso intero e binario.

2.2 I modelli di ottimizzazione

Tradurre un problema del mondo reale in una delle forme (2.1) o (2.3) significa costruirne un *modello*: identificare le variabili decisionali, esprimere l'obiettivo come funzione lineare di tali variabili e codificare i requisiti del problema come vincoli. Questa attività di *modellazione* è concettualmente distinta dalla *risoluzione* numerica, ed è proprio il livello a cui opera un linguaggio come ROOC.

Si consideri, a titolo di esempio, il classico problema dello *zaino* (*knapsack*): dato un insieme di n oggetti, ciascuno con un valore v_i e un peso w_i , e uno zaino di capacità C , si vuole selezionare il sottoinsieme di oggetti di valore complessivo massimo il cui peso non superi la capacità. Introducendo una variabile binaria $x_i \in \{0, 1\}$ che indica se l'oggetto i -esimo è selezionato, il modello si scrive

$$\begin{aligned} \max \quad & \sum_{i=1}^n v_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n w_i x_i \leq C, \\ & x_i \in \{0, 1\} \quad \forall i \in \{1, \dots, n\}. \end{aligned} \quad (2.4)$$

Si noti come la struttura del modello sia indipendente dai dati specifici (n , i valori v_i , i pesi w_i , la capacità C): la stessa formulazione descrive un'intera famiglia di istanze. La separazione tra la *struttura* del modello e i *dati* di una particolare istanza è un principio cardine dei linguaggi di modellazione, ROOC compreso, e ne motiva i costrutti come gli iteratori e le sommatorie indicizzate, descritti nel Capitolo 3.

2.3 Linguaggi e strumenti di modellazione esistenti

La modellazione di problemi di ottimizzazione è supportata da un'ampia gamma di strumenti, collocabili lungo uno spettro che va dalle rappresentazioni di basso livello, vicine al solver, fino ai linguaggi algebrici di alto livello, vicini alla notazione matematica.

All'estremo di basso livello, formati testuali come il *CPLEX LP* e l'*MPS* descrivono direttamente la matrice dei coefficienti di un modello: sono pensati come interfaccia tra software e solver, non offrono astrazioni (iteratori, costanti simboliche, strutture dati) e impongono di esplicitare ogni vincolo. ROOC è in grado di esportare in formato CPLEX LP, usato come interfaccia verso solver esterni. Più in alto si collocano i *linguaggi algebrici di modellazione* (AML), che esprimono i modelli con sommatorie, indici e parametri simbolici: il capostipite è **AMPL** [6], prodotto commerciale di cui **GNU MathProg** riprende un sottoinsieme libero [13], mentre **GAMS** ne è un altro storico rappresentante.

Un approccio più recente offre la modellazione come libreria di un linguaggio general-purpose, come **Pyomo** in Python [9] e **JuMP** in Julia [4]: ne eredita l'ecosistema, ma lega la sintassi del modello a quella del linguaggio ospite. **MiniZinc** adotta invece uno schema concettualmente vicino a quello di ROOC: un linguaggio di alto livello, indipendente dal solver, *compilato* in un formato intermedio (FlatZinc) risolvibile da diversi solver [15]. Va infine distinto il livello del *solver* vero e proprio: strumenti come **Gurobi** [8], **CPLEX** [11] o **HiGHS** [10] sono motori di risoluzione, non linguaggi di modellazione.

Questi strumenti costituiscono lo stato dell'arte con cui ROOC si confronta, e il loro punto di forza è la *generalità*. I linguaggi algebrici come AMPL o GAMS coprono un ampio ventaglio di classi di problemi (lineari, interi, non lineari) e si interfacciano con i solver più diversi, mentre motori come Gurobi e CPLEX offrono prestazioni di livello industriale, frutto di decenni di sviluppo e di ottimizzazioni sofisticate. Per problemi di grande dimensione o in contesti produttivi, dove ciò che conta è poter modellare qualunque cosa e risolverla nel minor tempo possibile, questi strumenti restano la scelta naturale.

Questa generalità richiede una sintassi e una semantica ricche, articolate e complesse, e dunque una curva di apprendimento ripida; molti di questi strumenti sono inoltre commerciali e a sorgente chiuso, oppure richiedono di padroneggiare un linguaggio di programmazione ospite, come Python per Pyomo o Julia per JuMP, e di allestire un ambiente di esecuzione. Per chi muove i primi passi nella modellazione, il rischio è che lo sforzo iniziale sia rivolto più allo strumento che ai concetti che si vorrebbero apprendere.

ROOC ha come obiettivo offrire un linguaggio capace di rappresentare con facilità modelli in grado di fare cose utili, senza per questo risultare difficile da comprendere. Da questo obiettivo discendono le sue priorità: una sintassi contenuta e vicina alla notazione matematica; una diagnostica che indica con precisione *dove* si è commesso un errore, così da accompagnare l'utente nella correzione; la possibilità di *estendere* il linguaggio con funzioni e costanti proprie quando i costrutti predefiniti non bastano; e, soprattutto, un orientamento all'*apprendimento* della modellazione e della risoluzione di problemi tramite la programmazione lineare. ROOC non compete dunque con gli strumenti citati sul terreno della generalità o delle prestazioni, ma occupa lo spazio, meno presidiato, di uno strumento pensato per imparare.

Esiste poi un secondo spazio poco presidiato, quello dell'ecosistema di Rust, in cui il panorama è più ristretto. La libreria di modellazione più diffusa è **good_lp**,¹ che offre un'interfaccia comoda verso numerosi solver ma rappresenta ogni espressione come combinazione affine delle variabili, non ammette operatori logici né funzioni come minimo, massimo o valore assoluto, che l'utente deve quindi linearizzare a mano. ROOC non si pone in alternativa a **good_lp**, che anzi impiega per

¹https://crates.io/crates/good_lp

accedere ai solver nativi, ma si colloca a un livello di astrazione superiore. La più recente **oximo**² ammette anche espressioni non lineari, la cui disponibilità dipende dal solver scelto, ma non prevede riformulazioni automatiche né operatori logici. **pldag**,³ infine, traduce formule logiche su variabili discrete in un sistema di disuguaglianze lineari da affidare a un solver esterno, ma è pensata per rappresentare vincoli più che per ottimizzare: non prevede variabili continue né riformula minimo, massimo o valore assoluto. Al momento della scrittura, dunque, nessuno strumento di modellazione dell'ecosistema Rust combina vincoli logici su modelli misti con la linearizzazione automatica di operatori logici, minimo, massimo e valore assoluto. Gli strumenti descritti in precedenza, d'altra parte, non possono colmare questa lacuna dato che richiedono un runtime esterno, come l'interprete Python o l'eseguibile di AMPL, oppure librerie native precompilate, e non possono quindi essere incorporati in un'applicazione Rust e compilati insieme a essa per qualunque target, WebAssembly compreso.

2.4 Cenni di teoria dei linguaggi e dei compilatori

ROOC è, a tutti gli effetti, un *compilatore*: traduce il linguaggio di modellazione in rappresentazioni progressivamente più vicine a una forma risolvibile da un solver. L'implementazione attuale arriva a un modello lineare in forma standard, ma l'architettura non è vincolata a tale destinazione. È quindi utile richiamare le fasi classiche di un compilatore [1], che forniscono la struttura portante del resto della tesi.

Analisi sintattica. Le prime fasi riconoscono la struttura del testo: l'*analisi lessicale* suddivide i caratteri in *token* e l'*analisi sintattica* (*parsing*) ne costruisce un *albero di sintassi astratta* (AST) secondo una grammatica. Accanto alle tradizionali grammatiche *context-free* (CFG), le *Parsing Expression Grammar* (PEG) [5] offrono una formalizzazione non ambigua, grazie alla scelta ordinata delle alternative, e integrano analisi lessicale e sintattica in un'unica grammatica: proprietà che le rendono adatte a ROOC (Capitolo 5).

Dalle analisi al codice. Superata l'analisi sintattica, l'*analisi semantica* verifica che il programma rispetti le regole del linguaggio (variabili dichiarate, operatori applicati a tipi compatibili, funzioni invocate correttamente) tramite il *sistema dei tipi* [16]; in ROOC questa fase comprende anche la *trasformazione* del modello (Capitolo 6). I compilatori passano poi attraverso una o più *rappresentazioni intermedie*, traducendole verso forme via via più vicine alla destinazione (*lowering*): in ROOC il lowering è la *linearizzazione* e la riduzione alla forma standard (Capitolo 7). Il ruolo del codice oggetto finale è infine svolto dal modello lineare, dato in pasto a un solver (Capitolo 8).

2.5 Posizionamento di ROOC

Alla luce di quanto esposto, ROOC si colloca come linguaggio di modellazione di alto livello, indipendente dal solver, con alcune scelte progettuali distintive:

²<https://oximo.dev>

³<https://crates.io/crates/pldag>

- una notazione dichiarativa vicina alla matematica, ma arricchita da un sistema di tipi che comprende strutture dati di alto livello (come grafi, tuple e iterabili) utili a modellare problemi di ottimizzazione combinatoria;
- un'architettura a compilatore con pipeline esplicita e *solver-indipendente*, sulla scia dell'approccio di MiniZinc, ma orientata alla programmazione lineare intera;
- un motore *pure Rust*, solver predefiniti compresi, utilizzabile come libreria sia attraverso il linguaggio sia attraverso una *fluent API*, che porta nell'ecosistema Rust i vincoli logici nei modelli misti e la loro linearizzazione automatica;
- l'esecuzione interamente lato client tramite la compilazione in WebAssembly [18], che elimina la necessità di un'infrastruttura server e rende lo strumento immediatamente accessibile da un browser;
- un taglio *didattico*: la possibilità di ispezionare ogni rappresentazione intermedia e di visualizzare passo dopo passo l'esecuzione del simplesso rende ROOC uno strumento utile non solo per risolvere modelli, ma anche per comprenderne il processo di risoluzione.

I capitoli successivi approfondiscono ciascuno di questi aspetti, a partire dalla descrizione del linguaggio dal punto di vista dell'utente (Capitolo 3).

Capitolo 3

Il linguaggio ROOC

Questo capitolo descrive il linguaggio ROOC dal punto di vista di chi lo utilizza: i costrutti che mette a disposizione e il loro significato. La realizzazione interna di tali costrutti è rimandata ai capitoli successivi.

3.1 Principi di design

Il progetto del linguaggio è guidato da alcuni principi di fondo. Il primo è la *vicinanza alla notazione matematica*: un modello scritto in ROOC deve ricordare il più possibile la formulazione che se ne darebbe su carta, con funzione obiettivo, vincoli e sommatorie indicizzate. Il secondo è la *separazione tra struttura del modello e dati*: la formulazione di un problema è indipendente dai valori numerici della singola istanza, che vengono forniti separatamente. Il terzo è la presenza di un *sistema di tipi statico*, che consente di individuare errori (variabili non dichiarate, operazioni tra tipi incompatibili, funzioni invocate in modo scorretto) prima ancora di tentare la risoluzione. Infine, il linguaggio è pensato per essere *estensibile*: oltre alla libreria di funzioni predefinite, l'utente può definire le proprie funzioni e le proprie costanti.

3.2 Struttura di un modello

Un modello ROOC è composto da una parte obbligatoria, che definisce l'obiettivo e i vincoli, e da due blocchi opzionali, `where` e `define`, che forniscono rispettivamente i dati e la dichiarazione del dominio delle variabili. La struttura generale è la seguente:

```
min/max <espressione obiettivo>
s. t.
    <vincoli>
where
    <dichiarazione delle costanti>
define
    <dichiarazione dei domini delle variabili>
```

Listing 3.1: Struttura generale di un modello ROOC.

Obiettivo. La prima riga dichiara l'obiettivo, introdotto dalla parola chiave `min` o `max` seguita dall'espressione da ottimizzare. In alternativa è ammessa la parola chiave `solve`, che richiede di trovare una qualunque soluzione ammissibile senza ottimizzare alcuna funzione obiettivo.

Vincoli. La sezione introdotta da `s.t.` (oppure `subject to`) contiene l'elenco dei vincoli. Ogni vincolo confronta due espressioni tramite un operatore di relazione (`<=`, `>=`, `=`, `<`, `>`). Un vincolo può essere preceduto da un nome (nella forma `nome: . . .`) e può essere seguito da una clausola di iterazione `for`, che lo replica per ogni valore degli indici, generando così un'intera famiglia di vincoli a partire da un'unica riga.

Blocco `where`. Il blocco opzionale `where` dichiara le *costanti* del modello, ciascuna con la sintassi `let nome = valore`. Il valore può essere una qualunque espressione del linguaggio: oltre a primitive come numeri, stringhe, array (anche multidimensionali) e grafi, sono ammesse espressioni più articolate, come chiamate di funzione o operazioni tra altre costanti. È qui che vengono tipicamente forniti i dati della specifica istanza.

Blocco `define`. Il blocco opzionale `define` dichiara il *dominio* delle variabili decisionali tramite la parola chiave `as`, come descritto nella Sezione 3.6. Anche le dichiarazioni di dominio possono essere indicizzate con una clausola `for`.

3.3 Esempio guida

Per illustrare in modo concreto i costrutti del linguaggio si introducono due esempi: il problema dello zaino e il problema del *dominating set* su grafo.

Il Listato 3.2 riprende il modello dello zaino già formalizzato nella Sezione 2.2. I dati dell'istanza (pesi, valori e capacità) sono dichiarati nel blocco `where`, mentre la variabile binaria x_i è dichiarata nel blocco `define`, una per ciascun oggetto.

```
max sum((value, i) in enumerate(values)) { value * x_i }
s.t.
    sum((weight, i) in enumerate(weights)) { weight * x_i } ≤ capacity
where
    let weights = [10, 60, 30, 40, 30, 20, 20, 2]
    let values = [1, 10, 15, 40, 60, 90, 100, 15]
    let capacity = 102
define
    x_i as Boolean for i in 0..len(weights)
```

Listing 3.2: Il problema dello zaino in ROOC.

Il secondo esempio (Listato 3.3) mostra l'espressività del linguaggio sui grafi. Dato un grafo G , il problema del *dominating set* chiede di selezionare il minimo insieme di nodi tale che ogni nodo sia selezionato oppure adiacente a un nodo selezionato. Il grafo è dichiarato come costante, e i vincoli vengono generati per ogni nodo tramite la clausola `for`.

```

min sum(u in nodes(G)) { x_u }
s.t.
    x_v + sum((_, u) in neigh_edges(v)) { x_u } ≥ 1 for v in nodes(G)
where
    let G = Graph {
        A → [B, C, D, E, F],
        B → [A, E, C, D, J],
        C → [A, B, D, E, I]
    }
define
    x_u, x_v as Boolean for v in nodes(G), (_, u) in neigh_edges(v)

```

Listing 3.3: Il problema del dominating set in ROOC.

3.4 Primitive e tipi di dato

Il linguaggio dispone di un insieme di tipi di dato primitivi, usati per le costanti, i risultati delle funzioni e i valori manipolati durante la trasformazione del modello. I tipi disponibili sono:

- i tipi numerici **Number** (numero in virgola mobile), **Integer** (intero con segno) e **PositiveInteger** (intero non negativo);
- **String**, le stringhe di caratteri;
- **Boolean**, i valori di verità **true** e **false**;
- **Graph**, **GraphNode** e **GraphEdge**, che rappresentano rispettivamente un grafo, un suo nodo e un suo arco;
- **Tuple**, sequenze di lunghezza fissa che possono contenere valori di tipi diversi;
- **Iterable**, le collezioni omogenee (per esempio gli array e gli intervalli) su cui è possibile iterare.

I tipi numerici sono distinti perché ciò consente al sistema dei tipi di essere più preciso: per esempio, la funzione **len** restituisce un **PositiveInteger**, garantendo staticamente la non negatività del risultato. La conversione tra tipi numerici avviene in modo automatico quando necessario.

Array. Gli array si scrivono tra parentesi quadre, come in `[10, 60, 30]`, e possono essere annidati per rappresentare matrici, come in `[[1, 2], [3, 4]]`. A livello di tipo, un array è un **Iterable** i cui elementi hanno tutti lo stesso tipo.

Grafi. Un grafo si dichiara con la parola chiave **Graph** seguita dall'elenco dei nodi e dei rispettivi archi uscenti. Ogni nodo ha la forma **Nodo** `-> [vicino1, vicino2, ...]`; a ciascun arco può essere associato un costo, indicato dopo il carattere `:`

```
let G = Graph {
  A → [B, C:2.5],
  B → [A],
  C → []
}
```

Listing 3.4: Dichiarazione di un grafo con costi sugli archi.

3.5 Iteratori, *block functions* e destructuring

Iteratori. Gli iteratori sono il meccanismo con cui il linguaggio esprime in modo compatto famiglie di vincoli, di variabili o di termini di una sommatoria. La forma generale è `for tupla in iterabile`; l'iterabile può essere un intervallo numerico, scritto con `..` (estremo superiore escluso) o `..=` (estremo superiore incluso), oppure una qualunque collezione, tipicamente prodotta da una funzione come `nodes(G)` o `enumerate(..)`. È possibile concatenare più iteratori separandoli con la virgola.

```
for i in 0..10      // interi da 0 a 9 (estremo superiore escluso)
for i in 0..=10     // interi da 0 a 10 (estremo superiore incluso)
for v in nodes(G)  // i nodi di un grafo
for (value, i) in enumerate(values) // coppie (elemento, indice)
for i in 0..n, j in 0..n // due iteratori concatenati
```

Listing 3.5: Esempi di iteratori.

Block functions e nozione di *scope*. Le *block function* sono funzioni di aggregazione il cui risultato è una singola espressione. Il linguaggio ne offre due forme, che si distinguono per la presenza o l'assenza di uno *scope*, cioè di un insieme di variabili di iterazione introdotte localmente.

Le *block scoped function* possiedono uno scope: hanno la forma `nome(iteratori){corpo}`, dove gli iteratori dichiarano una o più variabili visibili soltanto all'interno del corpo. Il corpo viene valutato una volta per ogni combinazione di valori degli iteratori, e i risultati sono combinati dall'operatore di aggregazione. Sono disponibili nelle varianti `sum`, `prod`, `min`, `max` e `avg`; per le espressioni logiche (Sezione 3.6) sono inoltre disponibili le aggregazioni `all`, `any` e `xor`.

```
sum((value, i) in enumerate(values)) { value * x_i }
```

Listing 3.6: Block scoped function: lo scope introduce la variabile `i` e l'elemento `value`.

Le *block function* prive di scope, invece, operano su un elenco di espressioni già note, senza introdurre nuove variabili: hanno la forma `nome{a, b, c}` e sono disponibili per `min`, `max`, `avg` e per le aggregazioni logiche `all`, `any` e `xor`. A questa famiglia appartiene anche `abs`, che calcola il valore assoluto e accetta esattamente una espressione, come in `abs { x - 5 }`.

```
max{ x_1, x_2, x_3 }
```

Listing 3.7: Block function senza scope: opera su espressioni elencate esplicitamente.

Nella forma con scope il numero di termini aggregati dipende dalla cardinalità degli iteratori e non è noto staticamente, mentre nella forma senza scope i termini sono quelli esplicitamente elencati. Per questo motivo le aggregazioni `sum` e `prod`, che hanno senso soprattutto su insiemi di cardinalità arbitraria, sono disponibili soltanto nella forma con scope.

Variabili composte. La notazione x_i , detta *variabile composta*, si scrive `x_i` e indica una variabile indicizzata. Gli indici possono essere identificatori o numeri (`x_i`, `x_0`) oppure espressioni racchiuse tra parentesi graffe (`x_{i+1}`). Durante la trasformazione del modello ogni combinazione concreta degli indici dà luogo a una variabile distinta del modello: iterando `x_i` su `i` in `0..3` si ottengono le variabili `x_0`, `x_1` e `x_2`. Gli indici vengono valutati nel contesto di iterazione e concatenati a formare il nome finale della variabile; questo meccanismo è descritto in dettaglio nel Capitolo 6. Un indice identificatore privo di valore nel contesto viene mantenuto letteralmente: `x_A` indica la variabile di nome `x_A` anche fuori da un'iterazione.

Destructuring. Il *destructuring* è la decomposizione di un valore strutturato, come una tupla, nelle sue componenti, ciascuna legata a un nome con un'unica scrittura anziché estraendole una a una; il simbolo `_` segnala una componente che non interessa e che viene ignorata. In ROOC il destructuring si applica laddove un iteratore produce tuple: nella dichiarazione delle variabili di iterazione se ne decompongono direttamente le componenti. Per esempio, in `(value, i) in enumerate(values)` ogni elemento prodotto da `enumerate` è una coppia (`elemento`, `indice`), e a ogni iterazione `value` assume il valore dell'elemento e `i` il suo indice; in `(_, u) in neigh_edges(v)`, invece, di ciascun arco interessa soltanto il nodo di arrivo `u`, mentre la prima componente viene ignorata.

3.6 Operatori e definizione delle variabili

Operatori. Sul piano delle espressioni il linguaggio mette a disposizione gli operatori aritmetici `+`, `-`, `*` e `/` e l'operatore unario di negazione. È supportata anche la *moltiplicazione implicita*: una scrittura come `2x_i` è interpretata come `2 * x_i`. Gli operatori sono *sovraccaricati* sui diversi tipi: oltre al consueto significato aritmetico sui tipi numerici, l'operatore `+` applicato a due stringhe ne produce la concatenazione. Sono implementate anche funzioni insiemistiche come `union`, `intersection` e `difference` (Sezione 3.7).

Operatori logici. Il linguaggio dispone inoltre di operatori *logici*, pensati per i problemi di ottimizzazione combinatoria in cui i vincoli hanno natura logica: la congiunzione `and` (alias `&&`), la disgiunzione `or` (alias `||`), la negazione `not` (alias `!`), l'implicazione `implies` (alias `->`), la doppia implicazione `iff` (alias `<->`) e la disgiunzione esclusiva `xor`. Gli operandi devono essere espressioni *booleane*: variabili dichiarate come `Boolean`, i literali `true` e `false` o altre espressioni logiche. Le aggregazioni `all`, `any` e `xor` viste nella Sezione 3.5 sono le controparti indicizzate di `and`, `or` e `xor`, e dispongono degli alias estesi `conjunction`, `disjunction` ed `exclusive_disjunction`.

Un'espressione logica può essere usata in due modi. Come *vincolo*: un vincolo privo di operatore di confronto è interpretato come l'asserzione che l'espressione logica sia vera. Come *valore*: un'espressione logica vale 0 oppure 1, e può quindi comparire all'interno di espressioni aritmetiche, per

esempio per contare quante condizioni risultano soddisfatte. Il modo in cui questi costrutti vengono ricondotti a vincoli lineari è descritto nel Capitolo 7.

Il Listato 3.8 ne mostra l'uso sul problema del *vertex cover*, un problema classico dell'ottimizzazione combinatoria: dato un grafo, si cerca il minimo insieme di nodi che tocchi ogni arco. Il vincolo è l'asserzione logica $x_u \text{ or } x_v$, replicata su tutti gli archi: per ogni arco almeno uno dei due estremi deve essere selezionato. Nell'istanza del listato la copertura minima è $\{A, D\}$, di cardinalità 2.

```
min sum(v in nodes(G)) { x_v }
s.t.
    x_u or x_v for (u, v) in edges(G)
where
    let G = Graph {
        A → [B, C],
        B → [D],
        C → [D],
        D → [E],
        E
    }
define
    x_v as Boolean for v in nodes(G)
```

Listing 3.8: Il problema del vertex cover in ROOC, con un vincolo logico.

La stessa espressività si applica agli esempi già visti: il vincolo del *dominating set* (Listato 3.3) può riscriversi in forma logica come $x_v \text{ or } \text{any}((_, u) \text{ in } \text{neigh_edges}(v)) \{ x_u \}$, leggibile come «il nodo è selezionato oppure lo è almeno uno dei suoi vicini».

Dichiarazione delle variabili. Nel blocco `define` ogni variabile decisionale viene associata a un dominio tramite la parola chiave `as`. I domini disponibili sono:

- `Boolean`: variabile binaria, che assume valore 0 o 1;
- `Real(min, max)`: variabile reale compresa tra i due estremi indicati; se gli estremi sono omessi (`Real`) la variabile è illimitata;
- `NonNegativeReal(min, max)`: variabile reale non negativa; se gli estremi sono omessi assume valori nell'intervallo $[0, +\infty)$;
- `IntegerRange(min, max)`: variabile intera compresa tra i due estremi, che in questo caso sono obbligatori.

Gli estremi possono essere espressioni, una dichiarazione come `x_i as NonNegativeReal(Fmin[i], Fmax[i]) for i in 0..f` associa così a ciascuna variabile `x_i` i propri estremi, letti dagli array `Fmin` e `Fmax`.

3.7 Libreria standard e funzioni definite dall'utente

Libreria standard. Il linguaggio offre una libreria di funzioni predefinite, utilizzabili nelle espressioni del modello. Le principali si possono raggruppare in tre famiglie. Le funzioni su *collezioni*

comprendono `len` (numero di elementi), `enumerate` (alias `enum`, che associa a ogni elemento il suo indice), `zip` (che combina più collezioni elemento per elemento), `range` (che genera un intervallo numerico) e le operazioni insiemistiche `union`, `intersection` e `difference`. Le funzioni sui *grafi* comprendono `nodes` (alias `V`), `edges` (alias `E`), `neigh_edges` (alias `N`, che restituisce gli archi incidenti a un nodo) e `neigh_edges_of` (alias `N_of`). La disponibilità di alias come `V`, `E` e `N` avvicina la notazione a quella usuale della teoria dei grafi.

Funzioni definite dall'utente. Oltre alla libreria standard, l'utente può estendere il linguaggio con proprie funzioni e costanti. Questa possibilità è offerta soprattutto in ambiente web (Capitolo 9), dove le funzioni si dichiarano in TypeScript, specificandone il nome, i tipi dei parametri, il tipo restituito e l'implementazione. Le funzioni così registrate diventano disponibili nelle espressioni del modello allo stesso modo di quelle predefinite, e partecipano alla verifica statica dei tipi. Il meccanismo che rende possibile questa integrazione, basato sulla compilazione in WebAssembly e su un'interfaccia comune tra Rust e JavaScript, è descritto nel Capitolo 9.

Capitolo 4

Architettura del compilatore

Prima di affrontare nel dettaglio le singole fasi di compilazione, questo capitolo ne offre la visione d'insieme: come la traduzione di un modello sia organizzata in una sequenza di trasformazioni e come questa sia realizzata da un'astrazione componibile, la *pipe*.

4.1 La pipeline multi-fase

La compilazione di un modello ROOC avviene attraverso una successione di *rappresentazioni intermedie*, ognuna delle quali raffina la precedente avvicinandola a una forma risolvibile. Ogni rappresentazione è un tipo di dato a sé stante: il codice sorgente (`String`) viene dapprima associato a un parser, da cui si ottiene il `PreModel`, frutto della sola analisi sintattica; il `PreModel` viene poi sottoposto ad analisi semantica e trasformato nel `Model`; quest'ultimo viene linearizzato in un `LinearModel`, ridotto alla forma standard (`StandardLinearModel`), tradotto nel *tableau* del simpleso e infine risolto. La Figura 4.1 illustra questa sequenza.

È importante osservare che questa sequenza non è rigida. La coda della pipeline è *configurabile*: a partire dal `LinearModel` è possibile, in alternativa al percorso verso il *tableau*, invocare direttamente uno dei solver, ottenendo immediatamente una soluzione senza passare per la forma standard. La scelta delle fasi da eseguire dipende quindi da ciò che si vuole ottenere, come si vedrà nella prossima sezione.

Oltre che a partire dal sorgente testuale, un modello può essere costruito direttamente nel codice Rust tramite la *fluent API* (Sezione 9.1).

4.2 L'astrazione *Pipe*

L'intera pipeline è costruita su una singola, semplice astrazione. Ogni trasformazione è una *pipe*, ossia un'unità che riceve un dato, lo elabora e ne restituisce un altro. Formalmente è un tipo che implementa il *trait* `Pipeable`, costituito da un unico metodo:

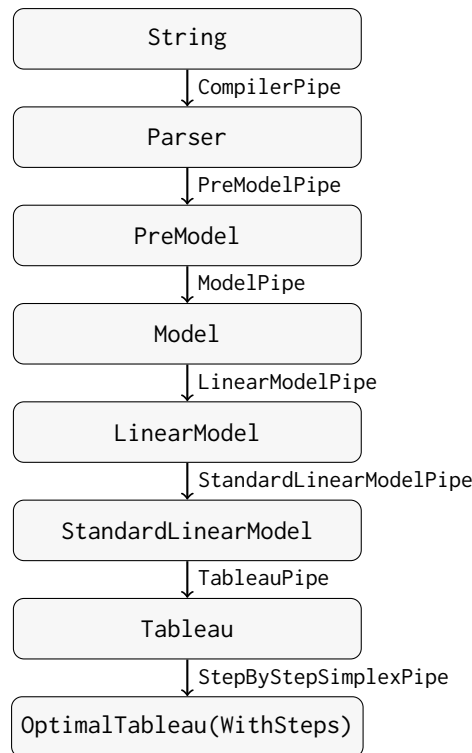


Figura 4.1: La pipeline di compilazione completa, dalla sorgente alla soluzione ottenuta tramite il semplice passo-passo. Ogni nodo è una rappresentazione intermedia; ogni arco è la *pipe* che effettua la trasformazione.

```

trait Pipeable {
  fn pipe(&self, data: &mut PipeableData, context: &PipeContext)
    → Result<PipeableData, PipeError>;
}
  
```

Listing 4.1: Il trait Pipeable.

I dati e i loro tipi. Tutti i dati che fluiscono tra le pipe sono rappresentati da un unico tipo enumerativo, `PipeableData`, le cui varianti corrispondono alle rappresentazioni intermedie della Figura 4.1 (`String`, `Parser`, `PreModel`, `Model`, `LinearModel`, `StandardLinearModel`, `Tableau` e le varie soluzioni). A ciascuna variante è associato un identificatore di tipo (`PipeDataType`), che permette di sapere quale dato una pipe abbia ricevuto.

Il contesto di esecuzione. Ogni pipe riceve, oltre al dato, un `PipeContext` condiviso, che mette a disposizione le costanti e le funzioni definite dall'utente. In questo modo le informazioni necessarie alla trasformazione (per esempio i dati dichiarati nel blocco `where` o le funzioni registrate dall'utente) sono accessibili in modo uniforme a tutte le fasi che ne hanno bisogno.

Le pipe concrete. La Tabella 4.1 elenca le pipe disponibili con i rispettivi tipi di ingresso e di uscita. Si noti come le prime sei realizzino il percorso “canonico” della Figura 4.1, mentre le ultime cinque siano *terminali* alternative che, a partire da un `LinearModel`, producono direttamente una soluzione delegando a un solver (Capitolo 8).

Pipe	Ingresso	Uscita
CompilerPipe	String	Parser
PreModelPipe	Parser	PreModel
ModelPipe	PreModel	Model
LinearModelPipe	Model	LinearModel
StandardLinearModelPipe	LinearModel	StandardLinearModel
TableauPipe	StandardLinearModel	Tableau
StepByStepSimplexPipe	Tableau	OptimalTableauWithSteps
RealSolver	LinearModel	RealSolution
MILPSolverPipe	LinearModel	MILPSolution
AutoSolverPipe	LinearModel	MILPSolution

Tabella 4.1: Le pipe disponibili e i tipi di dato che trasformano.

Esecuzione di una sequenza di pipe. Una sequenza di pipe viene eseguita da un `PipeRunner`, che le esegue sequenzialmente. Il dato iniziale viene passato alla prima pipe, il cui risultato è passato alla seconda, e così via. Un aspetto cruciale è che il `PipeRunner` non restituisce soltanto il risultato finale, ma l'intera *successione* dei risultati intermedi: ogni rappresentazione prodotta lungo il percorso viene conservata.

4.3 Estensibilità e visualizzazione passo-passo

L'astrazione descritta ha due conseguenze rilevanti, che motivano la scelta architetturale.

Estensibilità. Poiché ogni fase è semplicemente un tipo che implementa `Pipeable`, aggiungere una nuova trasformazione significa definire una nuova pipe, senza modificare quelle esistenti. Le pipe sono manipolate come oggetti-trait intercambiabili, e questo permette di comporre in modo flessibile: si possono inserire nuove fasi intermedie, sostituire un solver con un altro o prevedere percorsi alternativi. Anche la possibilità di terminare la pipeline con solver diversi (reale, binario, intero-binario, MILP o automatico) è una diretta conseguenza di questa impostazione.

Visualizzazione passo-passo. Il fatto che il `PipeRunner` conservi tutte le rappresentazioni intermedie, e non soltanto il risultato finale, è ciò che rende possibile il *taglio didattico* della piattaforma, che le espone una per una anziché mostrare la sola soluzione (Capitolo 9).

Capitolo 5

Analisi sintattica e rappresentazione intermedia

Questo capitolo apre la parte dedicata al *front-end* del compilatore, che comprende le prime fasi della traduzione: il riconoscimento della struttura del testo sorgente e la sua codifica in una rappresentazione interna su cui le fasi successive potranno lavorare.

5.1 Concetti teorici

Grammatiche e analisi sintattica. La sintassi di un linguaggio è descritta da una *grammatica*: un insieme di regole che stabiliscono quali sequenze di simboli costituiscono frasi valide. L'*analisi sintattica* (*parsing*) verifica che il testo sorgente rispetti tali regole e ne costruisce una rappresentazione strutturata. La formalizzazione più diffusa è quella delle grammatiche *context-free* (CFG), spesso scritte in notazione BNF, in cui ogni regola può prevedere più alternative. Le CFG possono però essere *ambiguous*, cioè ammettere più alberi di derivazione per la stessa frase, il che complica la costruzione del parser.

Parsing Expression Grammar. Le *Parsing Expression Grammar* (PEG) [5] offrono una formalizzazione alternativa, pensata direttamente in funzione del riconoscimento. La differenza fondamentale rispetto alle CFG è la *scelta ordinata*: quando una regola prevede più alternative, queste vengono provate nell'ordine in cui sono scritte e viene accettata la prima che ha successo. Questo rende una PEG intrinsecamente non ambigua. Inoltre una PEG integra l'analisi lessicale e quella sintattica in un'unica descrizione (*scannerless*), eliminando la necessità di un analizzatore lessicale separato: la gestione degli spazi e dei commenti è parte della grammatica stessa.

La rappresentazione intermedia. Il risultato del parsing è una rappresentazione ad albero del testo sorgente che ne cattura la struttura logica trascurando i dettagli sintattici superflui. Questa *rappresentazione intermedia* (IR) è la struttura dati su cui operano le fasi successive del compilatore. Una buona IR rispecchia da vicino i costrutti del linguaggio, così da rendere naturali le elaborazioni successive, e conserva le informazioni necessarie alla diagnostica, in particolare la posizione di ciascun elemento nel sorgente.

La scelta di una PEG per ROOC è motivata proprio da queste proprietà: la non ambiguità semplifica la definizione della grammatica, e l'approccio *scannerless* permette di descrivere in un solo file l'intera sintassi del linguaggio, comprese le sue particolarità come la moltiplicazione implicita.

5.2 Applicazione in ROOC: la grammatica PEST

ROOC definisce la propria sintassi tramite **PEST**, una libreria Rust che implementa le PEG. La grammatica è scritta in un file dedicato¹ e viene tradotta in un parser a tempo di compilazione.

La regola principale. La regola di partenza, `problem`, descrive la struttura complessiva di un modello: la funzione obiettivo, la sezione dei vincoli e i blocchi opzionali `where` e `define` (Listato 5.1).

```
problem = {
    SOI ~ nl* ~
    #objective = objective ~ nl+ ~
    (^"s.t." | ^"subject to") ~ nl+ ~
    #constraints = constraint_list ~
    (nl+ ~ ^"where" ~ #where = consts_declaration)? ~
    (nl+ ~ ^"define" ~ #define = domains_declaration)? ~
    nl* ~ EOI
}
```

Listing 5.1: La regola principale della grammatica.

Costrutto	Significato
#nome = . . .	<i>Tag</i> : assegna un nome a una sottoparte della regola, rendendola facilmente recuperabile in fase di costruzione dell'IR.
~	<i>Sequenza</i> : gli elementi devono comparire in successione, nell'ordine indicato.
	<i>Scelta ordinata</i> : le alternative sono provate da sinistra a destra e si accetta la prima che ha successo.
?, *, +	<i>Quantificatori</i> : l'elemento compare rispettivamente zero o una volta, zero o più volte, una o più volte.
nl	Regola ausiliaria che riconosce un'interruzione di riga (<i>newline</i>).
^". . . "	Stringa letterale <i>case-insensitive</i> : <code>^"s.t."</code> .
@{ . . . }	Regola <i>atomica</i> : trattata come un unico token terminale.
_{ . . . }	Regola <i>silenziosa</i> : non compare nell'albero risultante perché serve solo a strutturare la grammatica.
(. . .)	Raggruppamento: applica un operatore (per esempio ? o *) a un gruppo di elementi.
SOI, EOI	Inizio (<i>Start Of Input</i>) e fine (<i>End Of Input</i>) dell'input.

Tabella 5.1: Gli operatori della grammatica PEG/PEST utilizzati in ROOC.

¹La grammatica completa è disponibile nel repository del progetto: <https://github.com/Specy/rooc/blob/main/src/parser/grammar.pest>

La regola `problem` (Listato 5.1) impiega gran parte degli operatori delle PEG e delle estensioni introdotte da PEST, il cui significato è riassunto nella Tabella 5.1.

Regole atomiche, silenziose e scelta ordinata. Il Listato 5.2 mostra in concreto alcuni di questi operatori: la regola `objective_type` è *atomica* (`@`) e impiega la scelta ordinata (`|`) tra due stringhe *case-insensitive*, mentre `exp` è *silenziosa* (`_`) e combina opzionalità e ripetizione.

```
objective_type = @{ ^"min" | ^"max" }
exp = _{ unary_op? ~ exp_leaf ~ (binary_op ~ unary_op? ~ exp_leaf)* }
```

Listing 5.2: Esempi di regola atomica, silenziosa e di scelta ordinata.

Dall'albero di parsing all'IR. L'albero prodotto da PEST viene poi visitato e trasformato nella rappresentazione intermedia del modello.

5.3 Il PreModel e l'Intermediate Language

La rappresentazione intermedia di ROOC prende il nome di **PreModel**: è il modello così come risulta dalla sola analisi sintattica, prima dell'analisi semantica e della trasformazione. Esso raccoglie le quattro componenti di un modello, più un riferimento al codice sorgente originale, utile per la diagnostica.

La funzione obiettivo è rappresentata da un **PreObjective**, che ne indica il tipo (minimizzazione o massimizzazione) e l'espressione associata. Ogni vincolo è un **PreConstraint**, che contiene le due espressioni messe a confronto, il tipo di relazione, l'eventuale nome e le eventuali clausole di iterazione (`for`) che lo replicano. Sia l'obiettivo sia i vincoli contengono dunque delle *espressioni*, che costituiscono il cuore dell'IR.

Le espressioni: PreExp. Le espressioni sono rappresentate dal tipo enumerativo **PreExp**, le cui varianti rispecchiano da vicino i costrutti del linguaggio descritti nel Capitolo 3: valori primitivi, variabili semplici e composte, accessi ad array, chiamate di funzione, *block function* (con e senza *scope*) e operazioni binarie e unarie (Listato 5.3).

```
enum PreExp {
    Primitive(Spanned<Primitive>),
    Variable(Spanned<String>),
    CompoundVariable(Spanned<CompoundVariable>),
    ArrayAccess(Spanned<AddressableAccess>),
    FunctionCall(InputSpan, FunctionCall),
    BlockFunction(Spanned<BlockFunction>),
    BlockScopedFunction(Spanned<BlockScopedFunction>),
    BinaryOperation(Spanned<BinOp>, Box<PreExp>, Box<PreExp>),
    UnaryOperation(Spanned<UnOp>, Box<PreExp>),
    // ...
}
```

Listing 5.3: Versione semplificata dell'enumerazione PreExp.

Essendo un albero, una PreExp può contenere altre PreExp. Per esempio un'operazione binaria racchiude le due sotto-espressioni degli operandi. Le variabili composte, come `x_i`, sono rappresentate da un `CompoundVariable` che ne memorizza il nome e l'elenco delle espressioni usate come indice; analogamente, l'accesso a un array è descritto da un `AddressableAccess`.

5.4 Gestione e tracciamento degli errori

Un aspetto trasversale a tutte le fasi del compilatore è la capacità di segnalare gli errori indicando con precisione il punto del sorgente a cui si riferiscono.

A questo scopo, ogni elemento della rappresentazione intermedia è annotato con la propria posizione nel sorgente tramite un contenitore generico, `Spanned`, che incapsula un valore insieme alle informazioni sulla sua posizione (riga, colonna e lunghezza del frammento di testo). Come si è visto, gran parte delle varianti di `PreExp` avvolge i propri dati in uno `Spanned`, così che ogni nodo dell'albero conservi il punto del sorgente da cui proviene.

Grazie a questa annotazione, gli errori sono *posizionali*: ogni segnalazione è collegata al frammento di sorgente che l'ha generata, e può così essere presentata indicando dove l'errore ha origine. La stessa infrastruttura permette di tracciare non solo gli errori di parsing, ma anche quelli delle fasi successive, come gli errori di tipo e di trasformazione descritti nel Capitolo 6.

Capitolo 6

Analisi semantica: tipi e trasformazione del modello

Una volta che il sorgente è stato tradotto nella rappresentazione intermedia (Capitolo 5), il compilatore deve verificarne la correttezza e trasformarlo in una forma concreta. Queste due attività costituiscono l'analisi semantica.

6.1 Concetti teorici

Analisi semantica. Mentre l'analisi sintattica stabilisce se un testo è *ben formato*, l'analisi semantica stabilisce se esso è anche *dotato di significato* secondo le regole del linguaggio. Sono errori semantici, e non sintattici, l'uso di una variabile non dichiarata, l'applicazione di un operatore a operandi incompatibili o l'invocazione di una funzione con un numero errato di argomenti.

Sistemi di tipi. Lo strumento principale con cui si esprimono e si verificano queste regole è il *sistema dei tipi* [16]. Un tipo classifica i valori in base alla loro natura (numeri, stringhe, collezioni, e così via) e impone vincoli su come essi possono essere combinati. La verifica statica dei tipi (*type checking*) controlla tali vincoli prima dell'esecuzione, permettendo di respingere i programmi scorretti con un messaggio diagnostico anziché lasciarli fallire in un momento successivo. È utile distinguere fra il *valore* che un'espressione assume e il *tipo* che la descrive: il type checker ragiona sui tipi, ignorando i valori concreti.

Ambiti e tabella dei simboli. Per sapere a cosa si riferisce un nome, il type checker mantiene una *tabella dei simboli* che associa a ogni identificatore il suo tipo. Poiché i linguaggi prevedono costrutti che introducono nomi validi solo localmente (in ROOC, gli iteratori), la tabella è organizzata in *ambiti* (*scope*) annidati, tipicamente gestiti come una pila: l'ingresso in un nuovo costrutto introduce un ambito, la sua uscita lo rimuove, e la ricerca di un nome procede dall'ambito più interno verso quelli più esterni.

6.2 Applicazione in ROOC: il sistema dei tipi

In ROOC ogni valore manipolato a tempo di esecuzione è una *Primitive* (Capitolo 3); a essa corrisponde, sul piano dei tipi, una *PrimitiveKind*, che ne è la controparte statica. Mentre una *Primitive* contiene un valore concreto, una *PrimitiveKind* ne descrive soltanto la natura (Listato 6.1).

```
enum PrimitiveKind {
    Number, Integer, PositiveInteger,
    String, Boolean,
    Graph, GraphNode, GraphEdge,
    Iterable(Box<PrimitiveKind>),
    Tuple(Vec<PrimitiveKind>),
    Any,
    // ...
}
```

Listing 6.1: Versione semplificata dell'enumerazione *PrimitiveKind*.

Due varianti meritano attenzione. *Iterable* è *parametrica* sul tipo dei propri elementi: il tipo di `[1, 2, 3]` non è semplicemente “collezione”, ma “collezione di interi”. Analogamente, *Tuple* memorizza la sequenza dei tipi delle proprie componenti, eventualmente eterogenei. La variante *Any* rappresenta infine un tipo compatibile con qualunque altro, utile nei casi in cui una verifica troppo rigida sarebbe inopportuna.

Il legame tra espressioni e tipi è dato dal *trait* *WithType*, che ogni espressione implementa: il suo metodo `get_type` calcola la *PrimitiveKind* di un'espressione a partire dal contesto. Su questa base si esprimono le regole del linguaggio: l'applicabilità di un operatore a una coppia di tipi e la corrispondenza tra gli argomenti di una chiamata e la *firma* della funzione invocata. La violazione di tali regole produce un errore (per esempio un errore di operatore quando i due operandi hanno tipi incompatibili), trattato nella Sezione 6.4.

6.3 Il type checker

La verifica dei tipi è realizzata dal *trait* *TypeCheckable*, implementato da tutte le entità dell'IR (espressioni, vincoli, dichiarazioni, l'intero modello). Il suo metodo `type_check` attraversa ricorsivamente la struttura, segnalando il primo errore incontrato.

Il contesto di verifica. Lo stato della verifica è mantenuto da un *TypeCheckerContext*, che riunisce tre informazioni: una pila di *frame*, il *dominio statico* e una mappa dei token tipati. La pila di *frame* realizza gli ambiti annidati discussi in precedenza: ogni *Frame* è una tabella che associa nomi a *PrimitiveKind*, l'ingresso in un iteratore aggiunge un *frame* e la sua uscita lo rimuove. La ricerca del tipo di un nome scorre i *frame* dal più interno al più esterno, riproducendo le regole di visibilità lessicale. Il dominio statico raccoglie invece i tipi delle variabili decisionali dichiarate nel blocco `define`.

Dichiarazione dei nomi. Quando un iteratore introduce una nuova variabile, questa viene dichiarata nel frame corrente. La dichiarazione rifiuta i nomi che coincidono con parole riservate e, in modalità stretta, segnala la ridichiarazione di un nome già esistente. Regole analoghe valgono per gli indici delle variabili composte, che devono avere un tipo ammissibile (numerico, stringa o nodo di grafo) per poter formare il nome di una variabile del modello.

La mappa dei token tipati. Oltre a verificare la correttezza, il contesto popola una *mappa dei token* che associa a ogni posizione del sorgente il tipo dell'elemento corrispondente, sotto forma di `TypedToken` (posizione, tipo ed eventuale identificatore). Questa mappa non è necessaria alla compilazione in sé, ma è ciò che permette all'editor della piattaforma web di mostrare il tipo di un'espressione al passaggio del cursore e di fornire suggerimenti, come si vedrà nel Capitolo 9.

Un esempio di errore. Per rendere concreto il comportamento del type checker, si supponga di alterare il modello dello zaino (Listato 3.2) passando alla funzione `enumerate` la costante scalare `capacity` (un `Integer`) anziché un array. Poiché `enumerate` richiede una collezione, la verifica fallisce e produce il messaggio del Listato 6.2.

```
[WrongArgument] expected "Any[]", got "Integer"
  at 5:34 "capacity"
  at 5:24 "enumerate(capacity)"
  at 5:5 "sum((weight, i) in enumerate(capacity)) { weight * x_i }"
```

Listing 6.2: Errore di tipo con la relativa traccia posizionale.

Il messaggio riporta il tipo atteso (`Any[]`, cioè una collezione di elementi di tipo qualsiasi) e quello effettivamente ricevuto (`Integer`), seguiti da una *traccia* delle espressioni che racchiudono il punto incriminato, dalla foglia (`capacity`, riga 5 colonna 34) fino all'intero vincolo. Le posizioni provengono dalle annotazioni `Spanned` conservate nell'IR (Capitolo 5) e consentono di indicare con precisione il punto in cui l'errore ha origine.

6.4 Trasformazione PreModel → Model

Superata la verifica dei tipi, il `PreModel` viene trasformato in un `Model`. Questa fase è un vero e proprio *lowering*: le astrazioni di alto livello del linguaggio vengono risolte ed espanse, producendo un'espressione molto più semplice e concreta.

Preparazione del contesto. Il punto di ingresso raccoglie tutte le costanti disponibili (quelle predefinite, quelle eventualmente fornite dall'esterno e quelle dichiarate nel blocco `where`) e tutte le funzioni (predefinite e definite dall'utente), e costruisce un `TransformerContext`. Quest'ultimo è l'analogo, sul piano dei *valori*, del contesto di verifica: i suoi frame associano i nomi a valori concreti (`Primitive`) anziché a tipi, e un dominio raccoglie le variabili decisionali via via generate.

Da PreExp a Exp. Il risultato della trasformazione delle espressioni è il tipo `Exp` (Listato 6.3), notevolmente più semplice della `PreExp` di partenza. Tutti i costrutti di alto livello sono scomparsi: restano soltanto numeri, variabili (identificate dal loro nome) e operazioni.

```
enum Exp {
    Number(f64),
    Variable(String),
    BinOp(BinOp, Box<Exp>, Box<Exp>),
    UnOp(UnOp, Box<Exp>),
    // ...
}
```

Listing 6.3: Versione semplificata dell'enumerazione Exp.

La semplificazione è la conseguenza di tre operazioni svolte durante la trasformazione. Primo, le costanti e gli accessi agli array vengono *valutati*: un riferimento a una costante è sostituito dal suo valore. Secondo, le *block function* vengono *espansive*: una sommatoria su un iteratore diventa un albero di addizioni tra i termini concreti. Terzo, le variabili composte vengono risolte nelle variabili concrete del modello.

Espansione degli iteratori e delle variabili composte. Il cuore dell'espansione è una procedura ricorsiva di risoluzione degli insiemi di iterazione. Dato uno o più iteratori, essa valuta il rispettivo iterabile e, per ogni combinazione dei valori, introduce un nuovo frame in cui lega le variabili di iterazione ai valori correnti e invoca una funzione di *callback*. La stessa procedura serve in due contesti: replicare un vincolo dotato di clausola *for*, generando un vincolo distinto per ogni combinazione di indici, ed espandere il corpo di una *block scoped function* a ogni iterazione, combinando i risultati con l'operatore di aggregazione.

In entrambi i contesti, ogni variabile composta come *x_i* viene tradotta in una variabile del modello dal nome univoco: gli indici, ormai ridotti a valori concreti, sono convertiti in stringa secondo regole semplici (un numero diventa la sua rappresentazione decimale, un valore booleano diventa T o F, una stringa resta invariata, un nodo di grafo diventa il proprio nome) e concatenati con un *underscore*. Ogni variabile così ottenuta è registrata nel dominio come una *DomainVariable*, che ne ricorda il tipo, la posizione di origine e il numero di utilizzi.

I tre meccanismi (iterazione di un vincolo, espansione di una *block scoped function* e risoluzione delle variabili composte) agiscono di norma insieme. Si consideri il vincolo del Listato 6.4.

```
sum(j in 0..i + 1) { x_j } ≤ i + 1 for i in 0..3
```

Listing 6.4: Un vincolo che combina iterazione, sommatoria con *scope* e variabili composte.

La clausola *for i in 0..3* replica il vincolo per *i* = 0, 1, 2; per ciascun valore di *i* la *block scoped function* *sum* itera *j* sull'intervallo 0..i + 1, il cui estremo dipende dall'indice esterno, espandendo la sommatoria in un albero di addizioni, e infine ogni *x_j* è risolta nella corrispondente variabile del modello. Il risultato è la famiglia di vincoli concreti

$$\begin{aligned} i = 0 : & \quad x_0 \leq 1, \\ i = 1 : & \quad x_0 + x_1 \leq 2, \\ i = 2 : & \quad x_0 + x_1 + x_2 \leq 3. \end{aligned}$$

Errori di trasformazione. Gli errori che possono emergere in questa fase (variabili non dichiarate, operatori inapplicabili, firme di funzione non rispettate, indici di tipo errato, e così via) sono rappresentati da un `TransformError`. Come anticipato nel Capitolo 5, man mano che un errore si propaga verso l'esterno gli viene associata la posizione del costrutto che lo ha generato, costruendo una *traccia* che, a partire dal sorgente, permette di indicare con precisione dove l'errore ha avuto origine.

Capitolo 7

Linearizzazione e forma standard

Il Model prodotto dall'analisi semantica (Capitolo 6) è una descrizione concreta del problema, ma può ancora contenere costrutti non direttamente trattabili da un solver lineare. Questo capitolo descrive il *back-end* del compilatore: la traduzione del modello in una forma lineare e la sua successiva riduzione alla forma standard, prerequisito del metodo del simplesso.

7.1 Concetti teorici

Linearità. Un'espressione è *lineare* nelle variabili decisionali se è una combinazione lineare di queste, cioè una somma di termini ciascuno costituito da una variabile moltiplicata per un coefficiente costante. Sono lineari le somme e le differenze di espressioni lineari e il prodotto di una costante per una variabile; non lo è, invece, il prodotto di due variabili o la divisione per un'espressione che contenga variabili. Poiché i solver di programmazione lineare operano per definizione su modelli lineari, ogni costrutto non lineare deve essere eliminato o riformulato prima della risoluzione.

Riformulazione di minimo e massimo. Alcuni costrutti non lineari possono essere resi lineari introducendo *variabili ausiliarie* e vincoli aggiuntivi. È il caso del minimo e del massimo di un insieme di espressioni. Per rappresentare $t = \min(e_1, \dots, e_k)$ si introduce una nuova variabile t e si impone

$$t \leq e_i \quad \forall i \in \{1, \dots, k\}, \quad (7.1)$$

sostituendo l'occorrenza del minimo con t . Dualmente, per il massimo si impone $t \geq e_i$ per ogni i . Questa riformulazione è *unilaterale*: vincola t da un solo lato ed è quindi esatta soltanto quando è l'ottimizzazione stessa a spingere t contro il valore vero, come accade per un massimo che va minimizzato o per un minimo che va massimizzato.

Il metodo Big-M. Negli altri casi occorre imporre anche il vincolo dal lato opposto, che però deve valere solo per uno degli argomenti: quello che realizza davvero il minimo o il massimo. La tecnica classica per esprimere una scelta di questo tipo è il metodo *Big-M*: alla disuguaglianza si somma un termine $M(1 - \delta)$, dove δ è una variabile binaria e M una costante sufficientemente grande. Quando $\delta = 1$ il termine si annulla e il vincolo è attivo; quando $\delta = 0$ la costante M rende la disuguaglianza sempre soddisfatta, disattivandola di fatto. Un valore di M più grande del necessario peggiora però

la qualità numerica del modello, quindi conviene ricavare da estremi noti il più piccolo M sufficiente. Con intervalli $e_i \in [L_i, U_i]$ e $U_t = \max_i U_i$, il valore $t = \max(e_1, \dots, e_k)$ è descritto esattamente da

$$\begin{aligned} t &\geq e_i && \forall i, \\ t &\leq e_i + (U_t - L_i)(1 - \delta_i) && \forall i, \\ \sum_i \delta_i &= 1. \end{aligned} \tag{7.2}$$

La prima riga impone che t non sia inferiore ad alcun argomento; la seconda è il vincolo opposto, attivo solo per l'argomento scelto da δ_i , con $(U_t - L_i)$ nel ruolo di M ; la terza richiede che esattamente un argomento sia scelto. Il minimo si tratta in modo simmetrico, scambiando i versi delle disuguaglianze.

Riformulazione del valore assoluto. Il valore assoluto si riconduce al caso precedente osservando che $|e| = \max(e, -e)$: per rappresentare $t = |e|$ si impone

$$t \geq e, \quad t \geq -e, \tag{7.3}$$

con la stessa avvertenza sulla direzione di ottimizzazione; poiché $|e| \geq 0$, la variabile ausiliaria può inoltre essere dichiarata non negativa. Se l'intervallo di e attraversa lo zero e serve il valore esatto, un selettore binario disattiva uno dei due rami secondo lo schema Big-M appena visto. Se invece il segno di e è già noto, il valore assoluto coincide direttamente con e o con $-e$, senza variabili ausiliarie.

La forma standard. Il metodo del simplesso richiede che il problema sia espresso in *forma standard*: funzione obiettivo di minimizzazione, vincoli di sola uguaglianza e variabili tutte non negative. La riduzione a questa forma si ottiene con trasformazioni meccaniche. Un vincolo di disuguaglianza viene reso un'uguaglianza aggiungendo una variabile non negativa: una variabile di *slack* per i vincoli di tipo $\leq$ e una variabile di *surplus* per quelli di tipo $\geq$,

$$\sum_i a_i x_i \leq b \implies \sum_i a_i x_i + s = b, \quad s \geq 0. \tag{7.4}$$

Una variabile *libera* (priva di vincoli di segno e di estremi) viene sostituita dalla differenza di due variabili non negative, $x = x' - x''$ con $x', x'' \geq 0$. Più in generale, qualunque variabile che possa assumere valori negativi non può comparire tal quale nella forma standard: se limitata inferiormente da una costante finita negativa ($x \geq l$ con $l < 0$), può essere ricondotta alla non negatività tramite una traslazione ($x' = x - l \geq 0$) oppure, in modo equivalente ed uniforme, trattata imponendo i limiti come vincoli espliciti e sostituendo la variabile con la differenza di due variabili non negative. Infine, un problema di massimizzazione viene ricondotto a uno di minimizzazione osservando che $\max c^\top x = -\min(-c^\top x)$.

7.2 Applicazione in ROOC: da Model a LinearModel

La linearizzazione percorre ricorsivamente l'albero di ogni espressione e ne costruisce una rappresentazione lineare: una mappa che associa a ogni variabile il proprio coefficiente, più un termine costante. Addizione e sottrazione combinano le mappe termine a termine, la negazione moltiplica i

coefficienti per -1 , mentre moltiplicazione e divisione sono ammesse solo quando l'altro operando è costante. Un prodotto tra due espressioni che contengono variabili è intrinsecamente non lineare e viene segnalato come errore.

Analisi degli intervalli. Prima di generare i vincoli, il compilatore calcola per ogni variabile un intervallo conservativo, partendo dagli estremi dichiarati nel dominio e raffinandoli con le informazioni contenute nei vincoli del modello. Questi intervalli forniscono i valori L_i e U_i richiesti dalle formulazioni esatte della Sezione 7.1, senza ricorrere a costanti Big-M arbitrarie.

Direzione d'uso dei valori. Durante la visita, il compilatore tiene inoltre traccia di come ogni valore concorre all'ottimo: se l'ottimizzazione lo spinge verso il basso, verso l'alto, oppure se il valore deve essere esatto, come accade in un vincolo di uguaglianza. È questa informazione a decidere, per i costrutti non lineari, se basta la riformulazione unilaterale o se serve quella esatta. Il risultato è che ogni vincolo viene portato nella forma di una combinazione lineare confrontata con una costante e l'obiettivo in un vettore di coefficienti. Il risultato è un `LinearModel`: l'elenco delle variabili effettivamente usate, i loro domini, l'obiettivo e i vincoli lineari (`LinearConstraint`).

7.3 Linearizzazione di minimo, massimo e valore assoluto

Per minimo e massimo la formulazione è scelta in base alla direzione d'uso. Quando questa rende sufficiente la riformulazione unilaterale, il compilatore introduce una sola variabile reale e i vincoli (7.1), o i corrispondenti per il massimo: non servono variabili binarie e un problema lineare continuo resta tale. Quando invece è richiesto il valore esatto, viene applicata la formulazione (7.2), o la sua simmetrica per il minimo, con i coefficienti Big-M ricavati dagli intervalli calcolati. Se gli estremi finiti necessari non sono disponibili, la compilazione termina con un errore esplicito che indica le variabili prive di limiti, anziché introdurre una costante arbitraria.

Il valore assoluto segue lo stesso criterio: se il segno dell'argomento è noto viene sostituito direttamente da e o da $-e$; altrimenti si usa il solo epigrafo (7.3) quando la direzione d'uso lo consente, e il selettore binario negli altri casi.

7.4 Linearizzazione degli operatori logici

Gli operatori logici del linguaggio (Capitolo 3) vengono anch'essi eliminati in questa fase. Quando un'espressione logica è usata come valore numerico, ad esempio nell'obiettivo, viene applicata una *reifificazione*: la sottoespressione è sostituita da una variabile binaria (`$and_0`, `$or_0`, e così via), legata ai propri operandi da vincoli lineari. Poiché il sistema dei tipi garantisce valori pari a 0 o 1, sono applicabili le codifiche classiche della programmazione lineare intera. Per la congiunzione $z = e_1 \wedge \dots \wedge e_k$ si impone

$$z \leq e_i \quad \forall i, \quad z \geq \sum_i e_i - (k - 1), \quad (7.5)$$

mentre per la disgiunzione $z = e_1 \vee \dots \vee e_k$ si impone

$$z \geq e_i \quad \forall i, \quad z \leq \sum_i e_i. \quad (7.6)$$

La negazione non richiede alcuna variabile ausiliaria, perché $\neg e$ è l'espressione affine $1 - e$. L'implicazione $z = (a \Rightarrow b)$, essendo equivalente a $\neg a \vee b$, si codifica con $z \geq 1 - a$, $z \geq b$ e $z \leq 1 - a + b$; la doppia implicazione e la disgiunzione esclusiva sono trattate in modo analogo con quattro disuguaglianze ciascuna, mentre lo `xor` su più di due operandi è ridotto a una catena di `xor` binari.

Asserzioni logiche. Il caso più frequente, tuttavia, non è quello di un valore logico usato in un'espressione aritmetica, ma quello di un vincolo formato dalla sola formula, come `x_u` or `x_v`. Un vincolo di questo tipo è un'*asserzione*: non chiede di calcolare il valore di verità della formula, ma soltanto di escludere le soluzioni che la rendono falsa. Reificare la radice, cioè introdurre una variabile pari al valore della formula e fissarla a 1, sarebbe corretto ma inutilmente costoso: basta un insieme di vincoli le cui soluzioni ammissibili siano esattamente quelle che rendono vera la formula.

Il compilatore visita quindi la formula tenendo traccia del valore, vero o falso, che ciascuna sottoformula deve assumere. La negazione non genera vincoli: si limita a invertire il valore richiesto al proprio operando. Quando gli operandi sono variabili binarie o loro negazioni, l'asserzione si traduce in un unico vincolo lineare:

- $e_1 \wedge \dots \wedge e_k$ vera: $\sum_i e_i = k$; falsa: $\sum_i e_i \leq k - 1$;
- $e_1 \vee \dots \vee e_k$ vera: $\sum_i e_i \geq 1$; falsa: $\sum_i e_i = 0$;
- $a \Rightarrow b$ vera: $a \leq b$; falsa: $a - b = 1$.

La doppia implicazione e la disgiunzione esclusiva si riducono analogamente a un'uguaglianza, $a = b$ oppure $a + b = 1$.

Quando invece gli operandi sono a loro volta formule, si distinguono due casi. Nei casi *congiuntivi* (congiunzione vera, disgiunzione falsa, implicazione falsa) l'asserzione equivale a più asserzioni indipendenti sui figli: ad esempio $(e_1 \vee e_2) \wedge e_3$ è vera se e solo se sono veri sia $e_1 \vee e_2$ sia e_3 , e il compilatore prosegue ricorsivamente su ciascuno. Nei casi *disgiuntivi* (disgiunzione vera, congiunzione falsa, implicazione vera) basta invece che *almeno uno* dei figli assuma il valore richiesto, e l'asserzione non può essere spezzata. Per ogni figlio si introduce allora un *testimone* w_i , una variabile binaria vincolata in un solo verso: $w_i = 1$ implica che il figlio assuma il valore richiesto, mentre $w_i = 0$ non impone nulla. L'asserzione diventa $\sum_i w_i \geq 1$. Se il figlio è una semplice variabile e , il testimone è e stesso (o $1 - e$, se deve essere falsa) e non serve alcuna variabile nuova.

Per esempio, l'asserzione $(x \wedge y) \vee z$ introduce un solo testimone w per la congiunzione e genera i vincoli

$$w \leq x, \quad w \leq y, \quad w + z \geq 1. \quad (7.7)$$

Se $z = 0$, l'ultimo vincolo impone $w = 1$ e quindi $x = y = 1$; se $z = 1$, w è libero e la congiunzione non è vincolata. Il verso opposto, $w \geq x + y - 1$, non serve: il testimone non compare mai nell'obiettivo né in un'espressione aritmetica, quindi il solver può sceglierne liberamente il valore, e basta che $w = 1$ sia possibile solo quando il ramo è davvero vero. La reificazione esatta della stessa formula richiederebbe invece due variabili ausiliarie, una per la congiunzione e una per la disgiunzione, e sette vincoli. Nel caso più comune, come il vincolo di copertura $x_u \vee x_v$, non serve alcun testimone: il vincolo diventa direttamente $x_u + x_v \geq 1$, senza una variabile `$or_N`.

La reificazione esatta rimane invece necessaria quando il valore logico entra in un'espressione aritmetica: in quel caso la variabile ausiliaria coincide con il valore di verità in ogni soluzione ammissibile. In entrambi i casi la riformulazione avviene soltanto nel passaggio a `LinearModel`; il `Model` conserva la struttura logica originale, disponibile per eventuali risolutori futuri in grado di trattarla nativamente.

7.5 Trasformazione in forma standard

L'ultima trasformazione riduce il `LinearModel` a uno `StandardLinearModel`, applicando i passaggi teorici visti in precedenza.

Vincolo sulle variabili. La forma standard, così come è utilizzata dal simplesso, è definita per variabili *continue*. Per questo motivo la trasformazione richiede che tutte le variabili siano di tipo reale (con o senza vincolo di non negatività); in presenza di variabili intere o binarie restituisce un errore di dominio non valido. La risoluzione di modelli con variabili intere segue infatti un percorso diverso, descritto nel Capitolo 8.

I passaggi della trasformazione. La riduzione procede in quattro fasi. Anzitutto, per ogni variabile dotata di estremi finiti vengono aggiunti i vincoli espliciti che li impongono (un vincolo $\geq$ per il limite inferiore e un vincolo $\leq$ per quello superiore). In secondo luogo, ogni variabile non vincolata alla non negatività (ossia ogni variabile di tipo `Real`, sia essa illimitata o limitata da estremi che ammettono valori negativi) viene sostituita dalla differenza di due variabili non negative ($x = x' - x''$ con $x', x'' \geq 0$), riportando i relativi coefficienti sia nei vincoli del problema (inclusi i vincoli di delimitazione appena introdotti) sia nell'obiettivo. In terzo luogo, ogni vincolo viene normalizzato a un'uguaglianza secondo la (7.4), aggiungendo una variabile di slack o di surplus a seconda del verso della disuguaglianza. Infine, se il problema è di massimizzazione, i coefficienti dell'obiettivo vengono negati e viene registrato un apposito segnale, così da ricondurlo a una minimizzazione.

Il modello in forma standard. Il risultato è uno `StandardLinearModel`, che contiene l'elenco (ordinato) delle variabili, i coefficienti dell'obiettivo, i vincoli ridotti a uguaglianze, il termine costante dell'obiettivo (`objective_offset`) e un indicatore (`flip_objective`) che ricorda se l'obiettivo originale era di massimizzazione. Questi ultimi due valori permettono, una volta risolto il problema, di riconvertire il valore ottimo trovato in quello del problema di partenza.

A questo punto il modello è pronto per essere tradotto nel *tableau* del simplesso e risolto, come si vedrà nel prossimo capitolo.

Capitolo 8

Risoluzione dei modelli

Giunti in fondo alla pipeline di compilazione, il modello è pronto per essere risolto. Questo capitolo descrive le tecniche di risoluzione adottate da ROOC: il semplice a due fasi, implementato direttamente nel progetto a fini didattici; il solver `microlp`, contributo originale di questo lavoro e motore predefinito; infine i backend esterni che possono essergli affiancati.

8.1 Concetti teorici

Il metodo del semplice. Come accennato nel Capitolo 2, la soluzione ottima di un problema di programmazione lineare, se esiste, si trova in un vertice della regione ammissibile. Il *metodo del semplice* sfrutta questa proprietà spostandosi di vertice in vertice lungo gli spigoli del poliedro, scegliendo a ogni passo una direzione che migliora il valore della funzione obiettivo. Su un modello in forma standard, l'algoritmo lavora su un *tableau* e procede per *pivot*: a ogni iterazione seleziona una variabile entrante (quella che promette il miglioramento maggiore, individuata dai *costi ridotti*) e una variabile uscente (determinata dal *test del rapporto minimo*, che preserva l'ammissibilità), fino a quando non è più possibile migliorare e si è raggiunto l'ottimo.

Il metodo a due fasi. Il semplice richiede una base ammissibile di partenza, che non sempre è immediatamente disponibile. Il *metodo a due fasi* risolve questo problema introducendo, in una prima fase, delle *variabili artificiali* e minimizzando la loro somma: se il minimo raggiunto è zero, si è ottenuta una base ammissibile per il problema originale; la seconda fase ottimizza quindi la funzione obiettivo vera e propria a partire da tale base.

8.2 Applicazione in ROOC: il semplice a due fasi

ROOC include un'implementazione propria del metodo del semplice, costruita sopra la forma standard prodotta nel Capitolo 7. La struttura centrale è il **Tableau**, che mantiene i coefficienti dell'obiettivo, la matrice e il vettore dei termini noti dei vincoli, l'insieme delle variabili in base e il valore corrente. A partire dalla forma standard il solver costruisce il tableau, cercando una base ammissibile immediata e, quando questa non esiste, ricorrendo al metodo a due fasi descritto in precedenza; esegue quindi i vari passaggi del semplice, iterando i pivot, fino a raggiungere l'ottimo, rilevare l'illimitatezza del problema o toccare un limite massimo di iterazioni.

Risoluzione passo-passo. Oltre alla risoluzione diretta, il tableau offre una modalità *passo-passo* che, a ogni iterazione, ne registra lo stato prima del pivot insieme alle variabili entrante e uscente: è questa la funzionalità che alimenta la visualizzazione didattica della piattaforma (Capitolo 9). Trattandosi di un'implementazione orientata alla chiarezza più che alle prestazioni, questo simplesso è impiegato soprattutto a scopo illustrativo, mentre la risoluzione efficiente è affidata ai solver descritti nelle sezioni seguenti.

8.3 `microlp`: fork ed estensione a MILP

`microlp` è un solver pubblicato come *crate* Rust indipendente. Nasce come *fork* di `minilp`, un *crate* ormai archiviato che implementava il solo metodo del simplesso, limitato a variabili continue. Nell'ambito di questo lavoro il *crate* è stato ripreso e mantenuto, ed esteso con il supporto a variabili *interi* e *binarie*: da risolutore di sola programmazione lineare è diventato così un risolutore di programmazione lineare intera mista (MILP). Questa estensione costituisce uno dei contributi originali della tesi. La scelta di estendere un solver *pure Rust*, anziché affidarsi a risolutori nativi, risponde alla stessa motivazione di ROOC (Sezione 1.1) dato che `microlp` si compila insieme al resto del motore per qualunque target, ed è ciò che permette di risolvere modelli interi anche nel browser. Il *crate* ha peraltro una diffusione che va oltre questo progetto e sta raccogliendo l'interesse della comunità. È sviluppato come progetto *open source* su GitHub¹ ed è pubblicato su *crates.io*,² dove, al momento della scrittura, ha superato il milione di download; è inoltre utilizzato da `good_lp`, tra le più note librerie di programmazione lineare per Rust. Attorno al progetto si è formata una comunità che contribuisce attivamente, segnalando problemi e proponendo miglioramenti tramite *issue* e *pull request*.

Estensione al caso intero: `branch-and-bound`. Il simplesso risolve problemi a variabili continue, ma non garantisce soluzioni intere. La tecnica con cui `microlp` affronta l'integralità è il *branch-and-bound*: si risolve dapprima il *rilassamento continuo* del problema, ignorando i vincoli di integralità; se una variabile che dovrebbe essere intera assume un valore frazionario, si *ramifica* il problema in due sottoproblemi, imponendo nell'uno che la variabile sia minore o uguale alla parte intera inferiore del valore e nell'altro che sia maggiore o uguale a quella superiore. Il valore del rilassamento fornisce inoltre una *limitazione* (*bound*) che consente di potare i rami che non possono contenere soluzioni migliori di quella già trovata.

All'interno di ROOC, `microlp` è il solver predefinito: tratta tutti i domini previsti dal linguaggio, dai modelli puramente continui a quelli con variabili intere o miste, ed è quindi quello impiegato quando l'utente non ne indica uno esplicitamente. La risoluzione avviene costruendo un problema nel formato del solver: a ogni variabile del modello viene associata una variabile del solver, con il proprio dominio (reale, intero o binario) e i propri estremi; vengono quindi aggiunti i vincoli, tradotti nei rispettivi operatori di confronto, e infine si richiede la soluzione, da cui si estraggono i valori delle variabili e il valore ottimo.

¹<https://github.com/Specy/microlp>

²<https://crates.io/crates/microlp>

8.4 Backend esterni

Oltre a `microlp`, ROOC può affidare la risoluzione a numerosi solver esterni, integrati attraverso la libreria `good_lp`. Tutti ricevono lo stesso `LinearModel` e restituiscono la soluzione nello stesso formato, per cui il modello non dipende dal solver con cui viene risolto. Solo due sono abilitati per impostazione predefinita, perché scritti interamente in Rust e quindi compilabili anche in WebAssembly: `microlp` e `Clarabel`. Gli altri sono funzionalità opzionali del crate, che richiedono librerie native o programmi installati sul sistema: `HiGHS`, `CBC`, `SCIP`, `lp_solve` e `CPLEX`. Questi trattano sia problemi continui sia interi misti. La piattaforma web offre inoltre `HiGHS` tramite una sua compilazione in WebAssembly indipendente dal motore.

Capitolo 9

Dalla libreria alla piattaforma web

I capitoli precedenti hanno descritto il motore di ROOC: il linguaggio, il compilatore e i solver. Questo capitolo mostra come tale motore, scritto in Rust, venga reso disponibile come libreria riutilizzabile, in Rust e in TypeScript, e come piattaforma web interattiva con finalità didattiche.

9.1 La libreria Rust e la fluent API

Il motore di ROOC è pubblicato come *crate* Rust e può essere incorporato in un'applicazione come semplice dipendenza. Con i solver predefiniti, `microIp` e `Clarabel`, è interamente *pure Rust*: non richiede librerie native né runtime esterni, e si compila quindi insieme all'applicazione per qualunque target supportato dal compilatore Rust. Tramite `good_lp` sono inoltre disponibili, come funzionalità opzionali, solver nativi quali `HiGHS`, `CBC` e `SCIP`, che però non sono pure Rust e non possono essere compilati in `WebAssembly`.

Un modello può essere fornito alla libreria in due modi. Il primo è il linguaggio testuale descritto nei capitoli precedenti, compilato attraverso la pipeline di *pipe* (Capitolo 4). Il secondo è una *fluent API* [7], cioè un'interfaccia in cui il modello si costruisce concatenando chiamate di metodo, in una forma che si legge quasi come la sua formulazione matematica. È pensata per i casi in cui il modello nasce all'interno di un programma Rust, a partire da dati che il programma stesso produce o legge.

Il punto di partenza è il `ModelBuilder`. Le variabili si dichiarano con la macro `vars!`, oppure con il metodo `add_var` quando il nome o il dominio vanno calcolati, e ciascuna dichiarazione restituisce un riferimento di tipo `Var`. Sui riferimenti sono sovraccaricati gli operatori aritmetici e quelli logici (`&`, `|`, `^` e `!`), e la libreria offre funzioni analoghe alle *block function* del linguaggio: `sum`, `min`, `max`, `abs`, `all` e `any`. I vincoli si scrivono con la macro `constraint!`, che consente la notazione $a + b \leq c$ e le implicazioni $a \rightarrow b$: in Rust gli operatori di confronto restituiscono un valore booleano e l'operatore $\rightarrow$ non esiste, per cui senza la macro un vincolo non potrebbe essere scritto in questa forma. Infine, i metodi `with` e `with_all` aggiungono i vincoli, `maximize` e `minimize` fissano l'obiettivo e `solve_with` risolve il modello con il solver indicato.

Il Listato 9.1 mostra un modello che combina vincoli aritmetici e logici. Uno stabilimento deve decidere quali di quattro prodotti avviare, con le variabili binarie `y`, e in quale quantità produrli, con le variabili continue `x`, massimizzando il ricavo al netto dei costi di attrezzaggio. La materia prima disponibile è limitata, e ogni prodotto avviato va realizzato in un lotto compreso tra un minimo e un massimo, mentre un prodotto non avviato non può essere prodotto. A questi vincoli aritmetici se

ne aggiungono tre logici: il prodotto 0 può essere avviato soltanto insieme al prodotto 2, i prodotti 1 e 3 non possono essere avviati insieme, e avviare insieme i prodotti 0 e 1 fa risparmiare uno sconto b sull'attrezzaggio.

I dati dell'istanza sono normali variabili Rust, e le famiglie di vincoli, che nel linguaggio si scriverebbero con la clausola `for`, diventano iteratori passati a `with_all`. I vincoli logici si scrivono invece direttamente, con gli operatori `->`, `!` e `&`; lo sconto nell'obiettivo, $b * (y[0] \& y[1])$, usa una congiunzione come valore numerico, che la linearizzazione rappresenta con una variabile ausiliaria (Sezione 7.4).

```
use rooc::builder::sum;
use rooc::{constraint, vars, Microlp, ModelBuilder, Var, VariableType};

fn main() → Result<(), Box<dyn std::error::Error>> {
    // dati dell'istanza
    let r = [7.0, 5.0, 3.0, 6.0];    // ricavo per pezzo
    let m = [2.0, 1.0, 2.0, 3.0];    // materia prima per pezzo
    let k = [10.0, 6.0, 5.0, 5.0];   // costo di attrezzaggio
    let qmin = [2.0, 2.0, 2.0, 2.0]; // lotto minimo
    let qmax = [6.0, 5.0, 4.0, 5.0]; // lotto massimo
    let (mmax, b, n) = (13.0, 4.0, r.len());

    let mut model = ModelBuilder::new();
    let x: Vec<Var> = (0..n)
        .map(|i| model.add_var(format!("x_{i}"),
            VariableType::NonNegativeReal(0.0, qmax[i])))
        .collect();
    vars! { model => y[n]: bool; };

    let solution = model
        .maximize(sum((0..n).map(|i| r[i] * x[i] - k[i] * y[i]))
            + b * (y[0] & y[1]))
        .with(constraint!(materiale:
            sum((0..n).map(|i| m[i] * x[i])) ≤ mmax))
        .with_all((0..n).map(|i| constraint!(x[i] ≤ qmax[i] * y[i])))
        .with_all((0..n).map(|i| constraint!(x[i] ≥ qmin[i] * y[i])))
        .with(constraint!(accessorio: y[0] → y[2]))
        .with(constraint!(stampo: !(y[1] & y[3])))
        .solve_with(Microlp::new())?
        .into_solution()?;

    println!("ottimo = {}", solution.value());
    Ok(())
}
```

Listing 9.1: Un modello di produzione con vincoli aritmetici e logici, costruito con la fluent API.

Il `ModelBuilder` non ha una pipeline propria: al momento della risoluzione produce un `Model`, la stessa rappresentazione che il linguaggio testuale ottiene dopo l'analisi semantica, e da lì segue il percorso consueto della linearizzazione e del solver. Per questo tutte le riformulazioni del Capitolo 7,

dai vincoli logici a minimo, massimo e valore assoluto, sono disponibili anche a chi costruisce il modello nel codice. Nell'istanza del Listato 9.1 l'ottimo vale 28 e si ottiene avviando i prodotti 0, 1 e 2, con quantità 2, 5 e 2: il prodotto 2 viene avviato al solo lotto minimo perché lo richiede il prodotto 0, e la materia prima è utilizzata per intero.

9.2 Compilazione in WebAssembly e la libreria TypeScript

Il motore di ROOC è scritto in Rust, ma è pensato per essere eseguito anche nel browser. A questo scopo viene compilato in *WebAssembly* [18] tramite lo strumento `wasm-pack`. Una conseguenza importante di questa scelta è che l'intero processo di compilazione e risoluzione avviene *lato client*: non è richiesto alcun server.

Sopra il modulo WebAssembly è costruita la libreria TypeScript `@specy/rooc`, che ne espone le funzionalità con un'interfaccia tipizzata. La libreria distribuisce ROOC in forma precompilata, così che possa essere usato direttamente da TypeScript e JavaScript senza la toolchain di Rust, e mette a disposizione anche una propria fluent API, analoga a quella della Sezione 9.1. Attraverso la libreria è possibile analizzare e compilare un modello, ottenerne la versione lineare o la resa in LaTeX, comporre ed eseguire le pipe (Capitolo 4) e registrare funzioni definite dall'utente. Il Listato 9.2 mostra un esempio completo: si registra una funzione `sqr` e una costante, si definisce un piccolo modello che le utilizza, e lo si risolve componendo la pipeline fino al solver automatico, leggendone infine il valore ottimo.

```

import { makeRoocFunction, RoocRunnablePipe, Pipes, PrimitiveKind } from "
  @specy/rooc"
// funzione definita dall'utente: radice quadrata
const sqrt = makeRoocFunction({
  name: "sqrt",
  parameters: [["of_num", PrimitiveKind.Number]],
  returns: PrimitiveKind.Number,
  call: (num) => ({
    type: "Number",
    value: Math.sqrt(num.value)
  }),
})
// costante fornita dall'esterno
const constants = [{"base", { type: "Number", value: 16 }}]
// la funzione sqrt e la costante "base" sono usate nel modello
const source = `
max sqrt(base) * x
s.t.
  x ≤ 10
define
  x as NonNegativeReal`
// composizione della pipeline di esecuzione
const pipe = new RoocRunnablePipe()
const pipes = [
  Pipes.CompilerPipe, Pipes.PreModelPipe,
  Pipes.ModelPipe, Pipes.LinearModelPipe,
  Pipes.AutoSolverPipe
]
for (const p of pipes) {
  pipe.addPipeByName(p)
}
const result = pipe.run(source, constants, [sqrt])
const steps = result.value.unwrap()
const solution = steps[steps.length - 1].data
console.log("Valore ottimo:", solution.value)

```

Listing 9.2: Definizione di una funzione e di una costante, risoluzione di un modello con @specy/rooc.

Grazie a questa impostazione lo stesso motore è riutilizzabile in tre forme: come *crate* Rust, come pacchetto npm e come componente eseguibile nel browser.

9.3 L'IDE web e l'LSP nel browser

La piattaforma web è un'applicazione realizzata con SvelteKit e generata come sito statico, anch'essa quindi priva di componente server e ospitabile su un qualunque servizio di hosting statico. Il suo elemento centrale è un editor di codice, dotato di una definizione di linguaggio dedicata a ROOC per l'evidenziazione della sintassi (Figura 9.1).



Figura 9.1: L'editor della piattaforma web: a sinistra l'area di scrittura del modello, a destra il pannello con la pipeline di risoluzione, configurabile come sequenza di stadi.

I riquadri numerati nella Figura 9.1 indicano:

1. l'**editor di codice**, in cui si scrive il modello ROOC (qui il problema della dieta dell'Appendice A), con evidenziazione della sintassi e diagnostica in linea;
2. la **barra degli strumenti**, con le azioni per allegare file, mostrare il *runtime* e accedere al repository, alla condivisione e alla documentazione;
3. il pannello che rappresenta la *pipeline* come sequenza di stadi configurabili che l'utente può aggiungere e rimuovere;
4. i **controlli di esecuzione**, con la scelta della pipeline predefinita e i pulsanti per ripristinare ed eseguire il modello.

Funzionalità da Language Server. Sull'editor sono innestate alcune funzionalità tipiche di un *Language Server Protocol* (LSP), realizzate interamente nel browser appoggiandosi alla libreria WebAssembly. La segnalazione degli errori sfrutta la diagnostica del compilatore: gli errori di sintassi, di tipo e di trasformazione, corredati della loro posizione (Capitoli 5 e 6), vengono evidenziati direttamente nel codice. Il passaggio del cursore su un'espressione ne mostra il tipo, ricavato dalla mappa dei token tipati prodotta dal type checker (Capitolo 6). Sono inoltre disponibili il completamento automatico e la formattazione del codice.

9.4 Valore didattico

La caratteristica che distingue la piattaforma è la possibilità di ispezionare l'intero processo di risoluzione. Come descritto nel Capitolo 4, l'architettura a pipe conserva tutte le rappresentazioni intermedie; la piattaforma le rende visibili, mostrando il modello dopo l'analisi semantica, la sua resa in LaTeX, la versione linearizzata, la forma standard e, infine, la risoluzione (Figura 9.2).

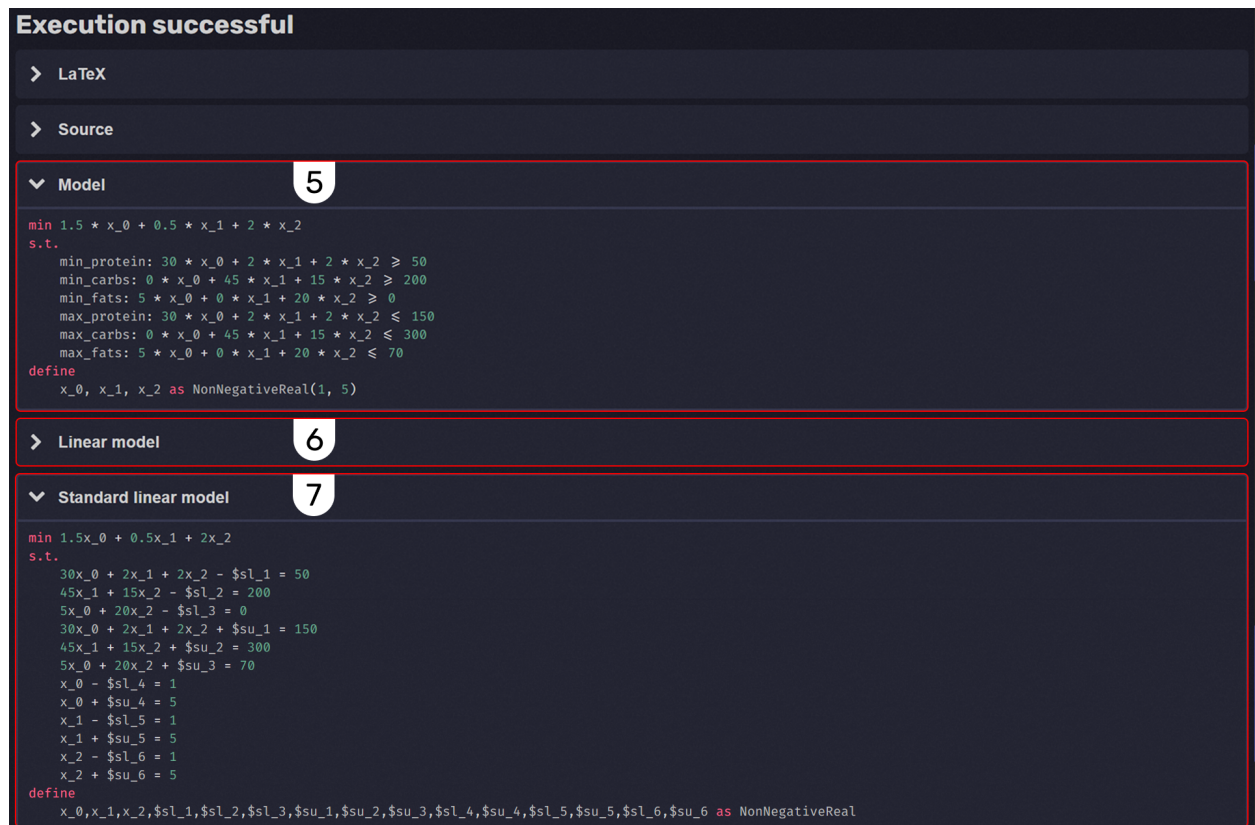


Figura 9.2: Le rappresentazioni intermedie esposte dopo l'esecuzione, dal modello alla forma standard.

I riquadri numerati nella Figura 9.2 indicano:

5. il **Model**, ottenuto dopo l'analisi semantica (Capitolo 6), in cui le astrazioni di alto livello sono risolte: le costanti sono valutate e gli iteratori espansi;
6. il **modello lineare**, in cui ogni espressione è ridotta a una combinazione lineare delle variabili (qui mostrato compresso dato che già lineare);
7. il **modello in forma standard**, con i vincoli ridotti a uguaglianze tramite variabili di *slack* e *surplus*, pronto per il simplesso.

In particolare, la modalità passo-passo del simplesso (Capitolo 8) permette di osservare l'evoluzione del *tableau* a ogni iterazione, con l'indicazione delle variabili entrante e uscente (Figura 9.3).

Simplex solver with steps **8**

Previous **1 / 2** Next

x_0	x_1	x_2	ssl_1	ssl_2	ssl_3	ssu_1	ssu_2	ssu_3	ssl_4	ssu_4	ssl_5	ssu_5	ssl_6	ssu_6		
0	0	0	0	0	0	-1/20	0	0	0	0	0	-2/5	19/10	0	-57/5	Basis
0	0	0	0	0	0	1/30	0	0	1	0	0	-1/15	1/15	0	18/5	ssl_4
0	0	0	0	0	0	0	0	0	0	0	1	1	-0	0	4	ssl_5
1	0	0	0	0	0	1/30	0	0	0	0	0	-1/15	1/15	0	23/5	x_0
0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	100	ssl_1
0	0	0	0	0	0	0	1	0	0	0	0	-45	15	0	60	ssu_2
0	0	0	0	0	0	-1/6	0	1	0	0	0	1/3	59/3	0	27	ssu_3
0	0	0	0	0	1	1/6	0	0	0	0	0	-1/3	-59/3	0	43	ssl_3
0	0	0	0	0	0	-1/30	0	-0	0	1	0	1/15	-1/15	0	2/5	ssu_4
0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	5	x_1
0	0	0	0	1	0	0	0	0	0	0	0	45	-15	0	40	ssl_2
0	0	1	0	0	0	0	0	0	0	0	0	0	-1	0	1	x_2
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	4	ssu_6

Pivoting: variable ssu_5 exits and ssl_4 enters

Optimal value: 6.04444 **9**

x_0	x_1	x_2	ssl_1	ssl_2	ssl_3	ssu_1	ssu_2	ssu_3	ssl_4	ssu_4	ssl_5	ssu_5	ssl_6	ssu_6		
0	0	0	1/20	0.00889	0	0	0	0	0	0	0	0	53/30	0	-6.04444	Basis
0	0	0	-1/30	0.00148	0	0	0	0	1	0	0	0	2/45	0	14/43	ssl_4
0	0	0	0	-1/45	0	0	0	0	0	0	1	0	1/3	0	28/9	ssl_5
1	0	0	-1/30	0.00148	0	0	0	0	0	0	0	0	2/45	0	57/43	x_0
0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	100	ssu_1
0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	100	ssu_2
0	0	0	1/6	-0.00741	0	0	0	1	0	0	0	0	19.77778	0	43.37037	ssu_3
0	0	0	-1/6	0.00741	1	0	0	0	0	0	0	0	-19.77778	0	26.62963	ssl_3
0	0	0	1/30	-0.00148	0	0	0	-0	0	1	0	0	-2/45	0	3.67407	ssu_4
0	1	0	0	-1/45	0	0	0	0	0	0	0	0	1/3	0	37/9	x_1
0	0	0	0	1/45	0	0	0	0	0	0	0	1	-1/3	0	8/9	ssu_5
0	0	1	0	0	0	0	0	0	0	0	0	0	-1	0	1	x_2
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	4	ssu_6

x_0	x_1	x_2	ssl_2	ssl_3	ssu_1	ssu_2	ssu_3	ssl_4	ssu_4	ssl_5	ssu_5	ssl_6	ssu_6
57/43	37/9	1	0	26.62963	100	100	43.37037	14/43	3.67407	28/9	8/9	0	4

10

Figura 9.3: La risoluzione passo-passo del simplesso e la soluzione finale del problema della dieta.

I riquadri numerati nella Figura 9.3 indicano:

- la **risoluzione passo-passo** del simplesso, con la navigazione tra le iterazioni e l'indicazione, a ogni *pivot*, delle variabili entrante e uscente;
- il **tableau ottimo**, raggiunto al termine delle iterazioni, con il relativo valore ottimo della funzione obiettivo;
- la **soluzione**, ossia i valori assunti dalle variabili del modello (x_0 , x_1 , x_2 e le variabili ausiliarie).

A completamento, la piattaforma offre una documentazione interattiva del linguaggio, con esempi eseguibili direttamente nell'editor. L'insieme di queste funzionalità fa sì che ROOC non sia soltanto uno strumento per risolvere modelli di ottimizzazione, ma anche un mezzo per comprendere come tali modelli vengano compilati e risolti. In particolare, può essere d'aiuto agli studenti dei corsi di Ricerca Operativa e di Ottimizzazione Combinatoria, offrendo loro un ambiente in cui imparare a formulare un modello e a risolverlo con un solver, osservando al tempo stesso ogni fase del procedimento.

Capitolo 10

Conclusioni e sviluppi futuri

10.1 Riepilogo dei contributi

Questa tesi ha presentato ROOC, un linguaggio per la modellazione di problemi di ottimizzazione e l'ambiente che lo accompagna. Il lavoro si è concentrato sul *motore*: un compilatore che traduce un modello, scritto in una notazione dichiarativa vicina alla matematica, in una forma lineare risolvibile da un solver.

I contributi principali possono essere riassunti come segue. È stato progettato un *linguaggio* dotato di un sistema di tipi e di costrutti di alto livello, come gli iteratori, le *block function*, gli operatori logici e i tipi di dato strutturati (Capitolo 3). È stato realizzato un *compilatore multi-fase* che, attraverso una sequenza di trasformazioni (Capitoli 5–7), riduce il modello alla forma standard, e la cui architettura a *pipe* (Capitolo 4) rende ogni fase componibile e ispezionabile. È stato esteso il solver *microlp*, a partire da un crate esistente limitato al simplesso, fino a coprire la programmazione lineare intera mista (Capitolo 8). Infine, il motore, pure Rust insieme ai suoi solver predefiniti, è stato reso disponibile come libreria riutilizzabile, anche attraverso una *fluent API* che porta nell'ecosistema Rust i vincoli logici nei modelli misti e la loro linearizzazione automatica, e come piattaforma web didattica (Capitolo 9).

10.2 Limitazioni

Il lavoro presenta alcune limitazioni note. Sul piano del linguaggio, gli operatori logici si applicano soltanto a espressioni booleane: non è possibile usare un confronto aritmetico come atomo logico (per esempio $(x + y \leq 5) \vee b$), costruito che richiederebbe la riformulazione dei confronti tramite costanti *big-M* e quindi un'analisi dei limiti delle variabili. Sul piano della risoluzione, il simplesso implementato nel progetto è orientato alla chiarezza didattica più che alle prestazioni, e la sua applicazione diretta è limitata ai modelli di dimensioni contenute; per i problemi più impegnativi ci si affida ai solver esterni. Infine, l'intera pipeline è oggi orientata ai modelli lineari, pur essendo l'architettura predisposta per ospitare in futuro forme diverse.

10.3 Sviluppi futuri

Diversi sono i possibili sviluppi. Il più immediato è l'estensione degli operatori logici ai confronti aritmetici (*vincoli indicatore*), che completerebbe la copertura dei problemi di ottimizzazione combinatoria modellabili in modo diretto. L'architettura a pipe si presta inoltre all'aggiunta di nuovi solver e di nuove fasi di trasformazione, anche verso forme non lineari. Sul fronte della piattaforma, l'ulteriore arricchimento degli strumenti di visualizzazione e della documentazione interattiva ne rafforzerebbe la vocazione didattica. ROOC si conferma così non solo come uno strumento per risolvere modelli di ottimizzazione, ma anche come una base estensibile su cui continuare a costruire.

Bibliografia

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006.
- [2] D. Bertsimas and J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, Belmont, MA, 1997.
- [3] G. B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, Princeton, NJ, 1963.
- [4] I. Dunning, J. Huchette, and M. Lubin. JuMP: A modeling language for mathematical optimization. *SIAM Review*, 59(2):295–320, 2017.
- [5] B. Ford. Parsing expression grammars: A recognition-based syntactic foundation. In *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '04)*, pages 111–122. ACM, 2004.
- [6] R. Fourer, D. M. Gay, and B. W. Kernighan. *AMPL: A Modeling Language for Mathematical Programming*. Thomson/Brooks/Cole, 2 edition, 2003.
- [7] M. Fowler. Fluent interface. <https://martinfowler.com/bliki/FluentInterface.html>, 2005.
- [8] Gurobi Optimization, LLC. Gurobi optimizer reference manual. <https://www.gurobi.com>, 2024.
- [9] W. E. Hart, C. D. Laird, J.-P. Watson, D. L. Woodruff, G. A. Hackebeil, B. L. Nicholson, and J. D. Sirola. *Pyomo — Optimization Modeling in Python*. Springer, 2 edition, 2017.
- [10] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.
- [11] IBM. IBM ILOG CPLEX optimization studio. <https://www.ibm.com/products/ilog-cplex-optimization-studio>, 2024.
- [12] S. Klabnik and C. Nichols. *The Rust Programming Language*. No Starch Press, 2019.
- [13] A. Makhorin. *GNU Linear Programming Kit, Reference Manual*. Free Software Foundation, 2017.
- [14] G. L. Nemhauser and L. A. Wolsey. *Integer and Combinatorial Optimization*. Wiley, New York, 1988.

- [15] N. Nethercote, P. J. Stuckey, R. Becket, S. Brand, G. J. Duck, and G. Tack. MiniZinc: Towards a standard CP modelling language. In *Principles and Practice of Constraint Programming (CP 2007)*, volume 4741 of *LNCS*, pages 529–543. Springer, 2007.
- [16] B. C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
- [17] Stack Overflow. 2025 Stack Overflow developer survey. <https://survey.stackoverflow.co/2025/>, 2025.
- [18] WebAssembly Community Group. Webassembly specification. <https://webassembly.org>, 2024.
- [19] L. A. Wolsey. *Integer Programming*. Wiley-Interscience, New York, 1998.

Appendice A

Un esempio di compilazione: il problema della dieta

Per illustrare in modo concreto l'intera pipeline di compilazione (Capitolo 4), si segue qui un singolo modello, il *problema della dieta*, lungo le sue rappresentazioni intermedie: dal sorgente al `Model` prodotto dall'analisi semantica, alla sua versione lineare (`LinearModel`) e infine alla forma standard (`StandardLinearModel`). Il modello impiega gran parte dei costrutti del linguaggio descritti nel Capitolo 3: costanti scalari e array (anche multidimensionali), funzioni della libreria standard (`enumerate`, `len`), sommatorie indicizzate, variabili composte, vincoli nominati e replicati tramite la clausola `for`, e variabili con estremi dipendenti dai dati. L'obiettivo è minimizzare il costo complessivo della dieta mantenendo ciascun nutriente entro i limiti previsti.

A.1 Sorgente

Il Listato A.1 riporta il modello così come viene scritto dall'utente. I dati dell'istanza (costi, fabbisogni dei nutrienti, porzioni minime e massime e la matrice `a` dei valori nutrizionali) sono raccolti nel blocco `where`, separati dalla struttura del modello.

```

min sum((cost, i) in enumerate(C)) { cost * x_i }
s.t.

min_{nutrient[j]}: //diet must have at least of nutrient j
    sum(i in 0..f) { a[i][j] * x_i } ≥ Nmin[j] for j in 0..len(Nmin)

max_{nutrient[j]}: //diet must have at most of nutrient j
    sum(i in 0..f) { a[i][j] * x_i } ≤ Nmax[j] for j in 0..len(Nmax)

where
    // Cost of chicken, rice, avocado
    let C = [1.5, 0.5, 2.0]
    // Min and max of: protein, carbs, fats
    let nutrient = ["protein", "carbs", "fats"]
    let Nmin = [50, 200, 0]
    let Nmax = [150, 300, 70]
    // Min and max servings of each food
    let Fmin = [1, 1, 1]
    let Fmax = [5, 5, 5]
    let a = [
        //protein, carbs, fats
        [30, 0, 5], // Chicken
        [2, 45, 0], // Rice
        [2, 15, 20] // Avocado
    ]
    // Number of foods
    let f = len(a)
    // Number of nutrients
    let n = len(Nmax)
define
    //bound the amount of each serving of food i
    x_i as NonNegativeReal(Fmin[i], Fmax[i]) for i in 0..f

```

Listing A.1: Il problema della dieta in ROOC.

A.2 Modello

L'analisi semantica e la trasformazione (Capitolo 6) producono il Model del Listato A.2, in cui tutte le astrazioni di alto livello sono state risolte. Le costanti sono state valutate e sostituite con i rispettivi valori; gli iteratori sono stati espansi: la clausola `for j in 0..len(Nmin)` replica i vincoli per i tre nutrienti, e il nome composto `min_{nutrient[j]}` diventa `min_protein`, `min_carbs` e `min_fats`. Le sommatorie `sum(i in 0..f)` sono diventate somme dei tre termini concreti, con i coefficienti letti dalla matrice `a`, e le variabili composte `x_i` sono state risolte in `x_0`, `x_1` e `x_2`, ciascuna con dominio `NonNegativeReal(1, 5)` (gli estremi `Fmin[i]` e `Fmax[i]` valutati). Si noti che a questo livello possono ancora comparire termini con coefficiente nullo, come `0 * x_0`.

```

min 1.5 * x_0 + 0.5 * x_1 + 2 * x_2
s.t.
    min_protein: 30 * x_0 + 2 * x_1 + 2 * x_2 ≥ 50
    min_carbs: 0 * x_0 + 45 * x_1 + 15 * x_2 ≥ 200
    min_fats: 5 * x_0 + 0 * x_1 + 20 * x_2 ≥ 0
    max_protein: 30 * x_0 + 2 * x_1 + 2 * x_2 ≤ 150
    max_carbs: 0 * x_0 + 45 * x_1 + 15 * x_2 ≤ 300
    max_fats: 5 * x_0 + 0 * x_1 + 20 * x_2 ≤ 70
define
    x_0, x_1, x_2 as NonNegativeReal(1, 5)

```

Listing A.2: Il Model dopo l'analisi semantica.

A.3 Modello lineare

La linearizzazione (Capitolo 7) riduce ogni espressione a una combinazione lineare delle variabili. I termini con coefficiente nullo vengono eliminati (per esempio $0 * x_0$ scompare da `min_carbs` e $0 * x_1$ da `min_fats`) e si adotta la moltiplicazione implicita. Il risultato è il `LinearModel` del Listato A.3, in cui restano soltanto le variabili effettivamente utilizzate in ciascun vincolo.

```

min 1.5x_0 + 0.5x_1 + 2x_2
s.t.
    min_protein: 30x_0 + 2x_1 + 2x_2 ≥ 50
    min_carbs: 45x_1 + 15x_2 ≥ 200
    min_fats: 5x_0 + 20x_2 ≥ 0
    max_protein: 30x_0 + 2x_1 + 2x_2 ≤ 150
    max_carbs: 45x_1 + 15x_2 ≤ 300
    max_fats: 5x_0 + 20x_2 ≤ 70
define
    x_0, x_1, x_2 as NonNegativeReal(1, 5)

```

Listing A.3: Il `LinearModel` dopo la linearizzazione.

A.4 Forma standard

L'ultima trasformazione (Sezione 7.5) riduce il modello alla forma standard del Listato A.4, richiesta dal semplice. Ogni disuguaglianza diventa un'uguaglianza con l'aggiunta di una variabile non negativa: una variabile sottratta per i vincoli di tipo $\geq$ e una aggiunta per quelli di tipo $\leq$ (qui con i nomi generati automaticamente `$sl_k` e `$su_k`). Inoltre gli estremi di ciascuna variabile, prima espressi dal dominio `NonNegativeReal(1, 5)`, diventano due vincoli espliciti (≥ 1 e ≤ 5), a loro volta normalizzati a uguaglianze. Poiché l'obiettivo era già di minimizzazione, non viene negato.

```
min 1.5x_0 + 0.5x_1 + 2x_2
s.t.
    30x_0 + 2x_1 + 2x_2 - $sl_1 = 50
    45x_1 + 15x_2 - $sl_2 = 200
    5x_0 + 20x_2 - $sl_3 = 0
    30x_0 + 2x_1 + 2x_2 + $su_1 = 150
    45x_1 + 15x_2 + $su_2 = 300
    5x_0 + 20x_2 + $su_3 = 70
    x_0 - $sl_4 = 1
    x_0 + $su_4 = 5
    x_1 - $sl_5 = 1
    x_1 + $su_5 = 5
    x_2 - $sl_6 = 1
    x_2 + $su_6 = 5
```

Listing A.4: Il modello in forma standard.

Ringraziamenti

Vorrei ringraziare il Prof. Matteo Spezialetti, il Prof. Stefano Smriglio, il Prof. Luca Forlizzi e il Prof. Claudio Arbib per i loro insegnamenti, che hanno alimentato la mia curiosità e la mia passione per l'informatica.

I miei genitori, mia sorella e la mia famiglia, per il loro supporto e incoraggiamento durante tutto il mio percorso di studi.

I miei amici e colleghi, per tutto il divertimento, per aver condiviso con me i loro interessi e per avermi mostrato prospettive e opportunità che altrimenti non avrei mai considerato.

I miei coinquilini, per tutte le giornate trascorse insieme, le cene condivise e le birre bevute.

Addio, e grazie per tutto il pesce.

Enrico Menichelli, L'Aquila, Ottobre 2026

